



Vision-based manipulation from single human video with open-world object graphs

Yifeng Zhu¹ · Arisrei Lim¹ · Peter Stone^{1,2} · Yuke Zhu¹

Received: 6 September 2025 / Revised: 1 February 2026 / Accepted: 31 March 2026 / Published online: 8 May 2026
© The Author(s) 2026

Abstract

This work presents an object-centric approach to learning vision-based manipulation skills from human videos. We investigate the problem of robot manipulation via imitation in the *open-world* setting, where a robot learns to manipulate novel objects from a single video demonstration. We introduce ORION, an algorithm that tackles the problem by extracting an object-centric manipulation plan as an *Open-World Object Graph* from a single RGB or RGB-D video, and then deriving a policy that conditions on the resulting plan. Our method enables the robot to learn from videos captured by daily mobile devices and generalize to deployment environments with varying visual backgrounds, camera angles, spatial layouts, and novel object instances. We systematically evaluate our method on both short-horizon and long-horizon tasks, using RGB-D and RGB-only demonstration videos. Across our real-world evaluations on varied tasks and demonstration modalities (RGB-D / RGB), we observe an average success rate of 74.4%, demonstrating the efficacy of ORION in learning from a single human video in the open world. Additional materials can be found on the [project website](#).

Keywords Robot Manipulation · Imitation From Human Videos · Object-Centric Learning · Open-World Imitation

1 Introduction

A critical step toward building robot autonomy is developing sensorimotor skills for perceiving and interacting with unstructured environments. Conventional methods for acquiring skills necessitate manual engineering and/or costly data collection (Dalal et al., 2021; Mandekar et al., 2020; Nasiriany et al., 2022; Zhang et al., 2023; Zhu et al., 2022). A promising alternative is teaching robots through human videos of manipulation behaviors situated in everyday sce-

narios. These methods have great potential to tap into the readily available source of Internet videos that encompass a wide distribution of human activities, paving the way for scaling up skill learning.

Prior work on learning from human videos has focused on pre-training representations and value functions (Chen et al., 2021; Nair et al., 2022; Ma et al., 2022; Wang et al., 2023; Xiong et al., 2021). However, they do not explicitly capture object interactions and states in 3D space where robot motions are defined. Consequently, they require separate teleoperation data for each set of objects in each location and even for each possible change in visual backgrounds, e.g., the scene background or lighting conditions (Zhu et al., 2023). In contrast, our goal is for a robot to learn to perform a task robustly in the “open world”, i.e., under varying visual and spatial conditions from a single human video and without explicit access to task-specific object models or robot demonstrations. Since our policy construction process uses actionless videos that are equivalent to state-only demonstrations in the problem of “imitation from observation” (Torabi, 2021), we refer to our problem setting as *open-world imitation from observation*.

Developing such a method in this setting becomes possible due to the recent advances in vision foundation models (Kir-

Y. Zhu and A. Lim have contributed equally to this work.

✉ Yifeng Zhu
yifeng.zhu@utexas.edu

✉ Arisrei Lim
arisrei@utexas.edu

Peter Stone
pstone@cs.utexas.edu

Yuke Zhu
yukez@cs.utexas.edu

¹ Computer Science, University of Texas at Austin, Austin, Texas, USA

² Sony AI, New York, USA

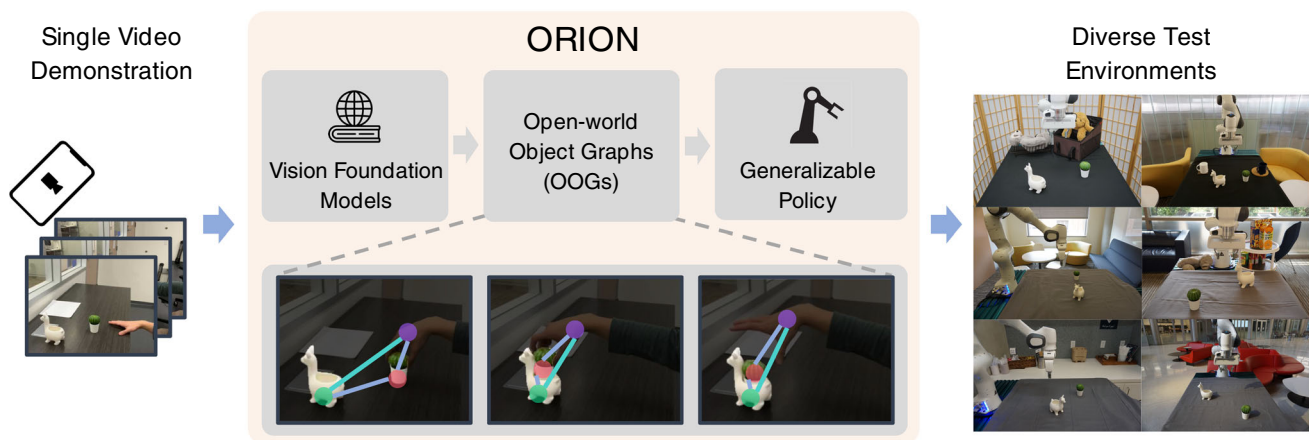


Fig. 1 Overview. We introduce ORION for tackling the problem of learning manipulation behaviors from single human video demonstrations. ORION first extracts a sequence of Open-World Object Graphs (OOGs), where each OOG models a keyframe state with task-relevant objects and hand information. Then, ORION leverages the

OOG sequence to construct a manipulation policy that generalizes across varied initial conditions, specifically in four aspects: visual background, camera shifts, spatial layouts, and novel instances from the same object categories

illov et al., 2023; Oquab et al., 2023). These models, pre-trained on Internet-scale visual data, excel at understanding open-vocabulary visual concepts and enable robots to recognize and localize objects in natural videos without known object categories or access to physical states. This work marks a major step toward achieving our vision of open-world imitation from observation, where a robot learns to interact with objects given *a single video* while generalizing to environments with different visual backgrounds and unseen spatial configurations during deployment. In this work, we consider using RGB or RGB-D video demonstrations where a person manipulates a small set of task-relevant objects, recorded with a stationary camera. These videos are actionless or state-only, as they do not come with any ground-truth action labels for the robot.

We introduce our method ORION, short for **Open-world ImitatiON**. Figure 1 visualizes a high-level overview of ORION. The core innovation lies in creating an object-centric spatiotemporal abstraction that effectively bridges the observational gap between human demonstration and robot execution. The design of ORION stems from our insight that many manipulation tasks can be sufficiently described by their object interactions, and that task completion depends on whether specific intermediate states, so-called *subgoals*, are reached. To capture the object-centric information in the video, we design a graph-based, object-centric representation, called Open-world Object Graphs (OOGs), to model the states of task-relevant objects and their relationships. An OOG has a two-level hierarchy. The high level consists of object nodes and a grasp node, where object nodes identify and localize the relevant objects by leveraging outputs from vision foundation models, while the grasp node encodes the

interaction information between the hand and objects, such as where to grasp. The low level consists of point nodes, which correspond to object keypoints, and the node features detail the motions of object keypoints in the 3D space.

ORION extracts a manipulation plan from the video as a sequence of OOGs and uses the plan to construct a generalizable policy. Experiments indicate that ORION constructs manipulation policies that are robust to conditions vastly different from those in the video. Using only an iPhone or iPad recording – or even synthetic videos generated by models such as Veo 2 (Deepmind, 2024) – of a task being performed in everyday environments (*e.g.*, an office or a kitchen), the resulting policies succeed in workspaces with drastically different visual backgrounds, camera angles, and spatial arrangements, and even generalize to manipulating unseen object instances of the same categories.

In summary, our contribution is threefold. 1) We formulate the problem of learning vision-based robot manipulation from a single human video in the open-world setting; 2) We introduce Open-world Object Graphs (OOGs), a graph-based, object-centric representation that models the states and relations of task-relevant objects; and 3) We present ORION, an algorithm that uses a single video to construct a manipulation policy, which generalizes to conditions that differ in four key ways: visual backgrounds, camera perspectives, spatial configurations, and new object instances.

2 Related Work

Learning Manipulation From Human Videos. Human videos offer a rich repertoire of object interaction behaviors,

making them an invaluable data source for manipulation. A large body of work has explored how to leverage human video data for learning robot manipulation (Wang et al., 2023; Xiong et al., 2021; Bahl et al., 2022; Kumar et al., 2023; Liu et al., 2018; Sharma et al., 2019; Smith et al., 2019; Xu et al., 2023), either through pre-training a single latent representation (Nair et al., 2022; Wang et al., 2023; Xu et al., 2023), learning an implicit reward function (Chen et al., 2021; Ma et al., 2022), or learning generative models that in-paint human morphologies (Ko et al., 2023; Bharadhwaj et al., 2023; Bahl et al., 2022; Bharadhwaj et al., 2023). However, they either require additional robot data from the target tasks or paired data between humans and robots. Our approach takes a different direction by tackling how a robot can imitate or learn from a single human video only: the robot does not rely on pre-existing data, models, or ground-truth annotations *in scenes* where video recording and robot evaluation take place. We refer to such a setting as *open-world imitation from observation*, where the robot is not programmed or trained to interact with the objects in the video *a priori* and the video data does not come with any robot actions. Our setting is closely related to the problem of “Imitation Learning from Observation” (Torabi, 2021), where state-only demonstrations are used to construct policies for physical interaction. However, this line of prior work assumes simulators of demonstrated tasks exist and physical states of the agents or objects are known (Pavse et al., 2020; Karnan et al., 2022; Torabi et al., 2019b, a, 2018). In contrast, our setting does not assume the digital replica of real-world tasks, and all the object information is perceived only through videos.

One-Shot Imitation Learning for Manipulation. Studies have delved into learning manipulation policies from one demonstration. A notable frame is one-shot imitation learning within a meta-learning framework proposed by Duan et al. (2017). While prior works on one-shot imitation learning have shown a robot performing new tasks from one demonstration, they require extensive in-domain data and a well-curated set of meta-training tasks beforehand, leading to significant data collection costs and restricted policy generalization at test time due to the tailored nature of the training.

An alternative approach involves using a single demonstration for initial guidance, refining the policy through real-world self-play (Di Palo and Johns, 2022; Haldar et al., 2023a, b; Johns, 2021; Valassakis et al., 2022). However, this approach mainly applies to reset-free tasks and struggles with scaling to multi-stage tasks where resetting to the task’s initial conditions does not come free. Our work aligns with these studies in using a single demonstration for learning manipulation, but stands out by not needing prior data or self-play. With just one single human video, our method constructs a policy that successfully completes the task, adapting to various visual and spatial differences from the task instance of video demonstration.

Open-World Imitation Learning in Robotics. Recent work (Park et al., 2025; Patel et al., 2025) has begun to bridge single-demonstration learning and learning from human videos, closely matching our “open-world” setting. Similarly to ORION, these works utilize the broad world knowledge of off-the-shelf models. However, they typically rely on stronger geometric inputs (*e.g.*, calibrated RGB-D or multi-view captures, explicit 3D hand poses, or pre-scanned object meshes) that are rarely available in casual consumer or Internet videos. These dependencies limit applicability to uncontrolled, single-view recordings. In contrast, our method requires only the demonstration video (RGB, or optionally RGB-D) with no pre-scanned object models.

Another recent approach, OKAMI (Li et al., 2024), most directly addresses the problem of “open-world imitation from observation”. However, OKAMI and ORION represent fundamentally different approaches to imitation. OKAMI’s method is *demonstrator-centric*, focusing on reconstructing the demonstrator’s body and hand motions and retargeting them onto a humanoid robot. In contrast, our method is *object-centric*, focusing on generating a plan that replicates the observed object trajectories, independent of the human’s specific pose. In addition, this means that ORION does not need a demonstration video showing the human’s entire body. By extracting object-centric Open-world Object Graphs (OOGs) with off-the-shelf vision foundation models, ORION recovers task-relevant object states and interaction cues directly from the video and constructs robot-executable plans that generalize across backgrounds, viewpoints, spatial layouts, and novel object instances.

Object-Centric Representations for Robot Manipulation.

The concept of object-centric representation has long been recognized for its potential to enhance robotic perception and manipulation by focusing on the objects within a scene. Prior works have shown the effectiveness of such representation in downstream manipulation tasks by factorizing visual scenes into disentangled object concepts (Devin et al., 2018; Migimatsu & Bohg, 2020; Tremblay et al., 2018; Tyree et al., 2022; Wang et al., 2019), but these works are typically confined to known object categories or instances. Recent developments in foundation models allow robots to access the open-world object concepts through pre-trained vision models (Kirillov et al., 2023; Oquab et al., 2023), enabling a wide range of abilities such as imitation of long-horizon tabletop manipulation (Shi et al., 2024; Zhu et al., 2022) or mobile manipulation in the wild (Stone et al., 2023). Building upon these advances, our work focuses on leveraging open-world, object-centric concepts in imitating manipulation behaviors from actionless human videos. We propose a graph-based representation called Open-world Object Graph (OOG), which allows a robot to imitate from a human video by leveraging the object-centric concepts. This proposed

representation shares a similar vein with prior works that factorize scene or task-relevant visual concepts into scene graphs (Huang et al., 2023; Kumar et al., 2023; Mo et al., 2019; Qureshi et al., 2021; Zhu et al., 2021). However, our representation is tailored to integrate open-world object concepts and enable generalization across different embodiments, specifically a human and a robot.

3 Problem Formulation

In this paper, we consider a vision-based, tabletop manipulation task, formulated as a finite-horizon Markov Decision Process (MDP) described by a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, H, R, \mu \rangle$, where $\mathcal{S}$ is the state space of raw sensory data and robot proprioception, $\mathcal{A}$ is the action space of low-level robot commands, $\mathcal{P} : \mathcal{S} \times \mathcal{A} \mapsto \mathcal{S}$ is the transition dynamics, H is the maximal task horizon, R is the sparse reward function, and μ is the initial state distributions of a task. We focus on the case where task reward functions are defined based on the object relations (contact and non-contact relations) between a small set of *task-relevant objects*. For example, a contact relation change occurs when a mug is placed on top of a coaster. A non-contact relation change occurs when a cup is angled to pour liquid into a bowl. A reward function returns 1 if all object relations of a task are satisfied and 0 otherwise. We also focus on manipulation tasks involving rigid-body objects with minimal occlusion in the demonstration video and rollout environment. The primary objective of solving a manipulation task is to find a visuomotor policy π that maximizes the expected task success rate from a wide range of initial environment configurations characterized by μ , where the states vary across the following four dimensions: 1) changing visual backgrounds, 2) different camera angles, 3) different object instances from the same categories, and 4) varied spatial layouts of the task-relevant objects.

We assume a robot does not have direct access to the ground-truth task reward or the physical states of task-relevant objects. In addition, we assume a single *actionless* video (Ko et al., 2023; Wen et al., 2023). V is provided as a *state-only* demonstration consisting of an RGB or RGB-D video stream of a person manipulating task-relevant objects, captured as an arbitrarily long sequence of images using a stationary camera. In addition, V involves a manipulation sequence where the object relations among task-relevant objects or hands change (e.g., an object is gripped, then placed on top of another object, or an object is tilted into a pouring position over another object). In this scenario, however, the robot is not pre-programmed to have access to ground-truth categories and locations of the task-relevant objects in V . We refer to this challenging setting as “open-world” (Joseph et al., 2021), as the robot must imitate from V while not pre-programmed or trained to interact with the objects in V . To allow a robot to operate in this “open-

world” setting, we assume access to common-sense world knowledge through vision-language models pre-trained on internet-scale data.

For evaluation, we adopt the following procedure. Given a single video V that matches the object relations of a task instance drawn from μ , the model’s performance is measured by the average rewards received when evaluating new task instances drawn from the same μ . If the manipulation in V does not result in object relations that satisfy the task’s requirements (e.g., the task specifies pouring juice into a cup, but the video pours juice on the floor), it is not considered a valid demonstration and thus should not be used for evaluation.

4 Method

In this section, we describe our method ORION (**Open-world video ImitatiON**). ORION is an algorithm that allows a robot to mimic human execution of a task, given a single demonstration video V . To effectively construct a policy π from V , ORION employs a learning objective based on an object-centric prior. The goal is to create a policy π that directs the robot to move objects along trajectories that mimic the directional and curvature patterns observed in V , relative to the objects’ initial and final positions. This objective is based on the observation that objects are likely to achieve target configurations by moving along trajectories similar to those in V . Key to ORION is generating a manipulation plan from V , which serves as the spatiotemporal abstraction of the video that guides the robot to perform a task. A plan is an arbitrarily long sequence of object-centric keyframes, where each keyframe specifies an initial or a subgoal state captured in V . We first introduce our formulation of the object-centric representation of a state, Open-world Object Graph (OOG), used in ORION, and then describe the algorithm that constructs a robot policy given a human video.

4.1 Open-world Object Graph

At the core of our approach is a graph-based, object-centric representation, Open-world Object Graphs (OOGs). OOGs use open-world vision models to model visual scenes with task-relevant objects and manipulators (e.g., a human hand or gripper) such that they naturally exclude the distracting factors in visual data and localize the task-relevant objects regardless of their spatial locations (see Section 4.2).

We denote an OOG as $\mathcal{G}$, which consists of object, gripper, and point nodes. At the high level, each object node corresponds to a task-relevant object described by the output of open-world vision models. Every object node comes with node features, consisting of colored 3D point clouds derived from RGB and depth observations. These node features indi-

cate both what and where objects are and also represent their geometry information. For RGB-only videos, an object node additionally includes an estimated object mesh, along with depth estimated from monocular depth estimation models. Next, to inform the robot where to interact with objects (*e.g.*, where to grasp), we introduce the specialized “grasp node”, which stores the interaction cues such as contact points and grip status (open or closed) that can be directly mapped to the robot end-effector during execution. At the low level, each point node corresponds to a keypoint that belongs to a task-relevant object. Every point node comes with an object-centric feature trajectory. This trajectory contains a list of features describing the object’s state throughout the video, such as a list of 3D points or 6-DOF poses. The information in this trajectory explicitly models how an object should be moved during a manipulation task. In the rest of the paper, by motion features of a point node in G_l , we mean the object-centric feature trajectory between keyframe l and $l + 1$.

In an OOG, all the object nodes and the hand node are fully connected, reflecting real-world spatial relationships. Additionally, the edges are augmented with a binary attribute, indicating whether two objects or an object and the hand are in contact. This attribute allows our developed algorithm to check the set of contact relations that are satisfied, retrieving the matched OOG from the generated plan (see Section 4.2). As for the low-level point nodes, they are connected to their respective object node, indicating a belonging relationship. In the rest of the paper, we denote node entities from human videos with a superscript V , and denote the ones from the robot rollout with a superscript Ro . Table 1 in the appendix also summarizes the variables needed to define an OOG.

4.2 Manipulation Plan Generation From V

We describe the first part of ORION (see Figure 2 and Algorithm 1), where our method automatically annotates the video and generates a manipulation plan from a given human video, V . In this paper, a manipulation plan is a spatio-temporal abstraction of V that centers around the object states and their motions over time, demonstrating how a task should be completed. Our core insight is that we can cost-effectively model a task with object locations at some keyframe states where the set of satisfied contact relations is changed, and abstract a majority of intermediate states into object motion trajectories (3D translations or 6-DOF pose trajectories). Concretely, a plan is represented as a sequence of OOGs, $\{G_l\}_{l=0}^L$, which corresponds to $L + 1$ keyframes in V , with G_0 representing the initial state. Note that ORION processes demonstration videos differently depending on whether depth information is available. We refer to these as the *RGB-D* case (with RGB and depth) and the *RGB* case (with RGB only).

Tracking task-relevant objects. ORION first localizes task-relevant objects in the video V (Lines 1-3). To begin, ORION samples frames from V and passes these frames to a VLM such as GPT-4o. The VLM is then prompted to return a list of strings describing each task-relevant object, while also indicating whether an object is being manipulated (details in Appendix E). This list is then used as an input to an open-world vision model, Grounded-SAM (Liu et al., 2023), to annotate video frames with segmentation masks of the task-relevant objects. In practice, open-world vision models are computationally demanding, so we reduce the computation by exploiting object permanence to track the objects (Lines 4-10). Specifically, ORION annotates the first video frame with Grounded-SAM, and then propagates the segmentation to the rest of the video using a Video Object Segmentation Model, Cutie (Cheng et al., 2023). Using 1 NVIDIA A5000, running Grounded-SAM takes 0.4 seconds per frame. However, by passing the first frame’s segmentation to Cutie, we can get this down to 0.024 seconds per frame, making processing demonstrations much faster. In the RGB case, ORION tracks an object’s 6-DOF trajectory with FoundationPose (Wen et al., 2024), modified to work with an RGB-only video, by searching across object pose hypotheses within a range of depth. The input to FoundationPose includes estimated depth, per-frame object masks, and an estimated object mesh, which is generated by InstantMesh (Xu et al., 2024) (implementation details in Appendix A.2).

Discovering keyframes. After annotating the locations of task-relevant objects, ORION tracks their motions across the video to discover K keyframes based on the velocity statistics of object motions (Line 12). This design is based on the observation that changes in object contact relations due to manipulation are often accompanied by sudden changes in object motions (*e.g.*, transitioning from free space motion to grasping an object). However, keeping full track of object point motion using techniques like optical flow estimation often results in unsatisfactory tracking quality, largely due to occlusions during manipulation. ORION uses a track-any-point (TAP) model, namely CoTracker (Karaev et al., 2023), to track a subset of points in a long-term video with explicit occlusion modeling, which has been successfully applied to track object motions in robot manipulation (Vecerik et al., 2023; Wen et al., 2022). Specifically, ORION first samples keypoints within the object segmentation of the first frame and track the trajectories across the video. The changes in velocity statistics are straightforward to detect based on the TAP trajectories, where we discover the keyframes using a standard unsupervised changepoint detection algorithm (Killick et al., 2012). In the RGB case, we find that setting $K = 2$, where the first keyframe is the first video frame and the last keyframe is the last video frame, still results in an OOG sequence that sufficiently describes the object motions in V .

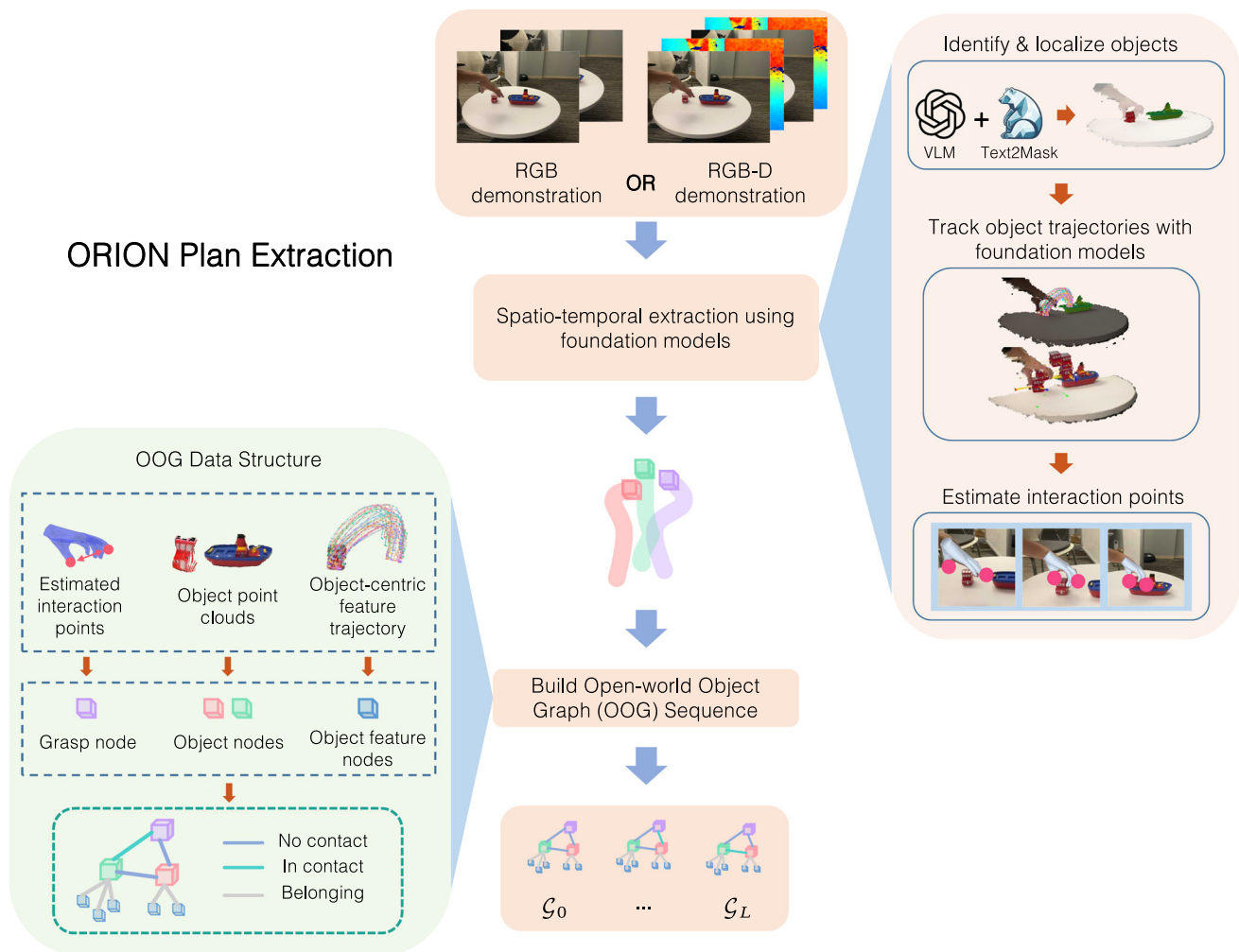


Fig. 2 Overview of plan generation in ORION. ORION generates a manipulation plan from a given video V in order for subsequent policies to synthesize actions. ORION first tracks objects and keypoints across the video frames. Then, keyframes are identified based on the veloc-

ity statistics of the keypoint trajectories. Finally, ORION generates an Open-world Object Graph (OOG) for every keyframe, resulting in a sequence of OOGs that serves as the spatiotemporal abstraction of the video. The figure is best viewed in color

Generating OOGs from V . Once ORION discovers the keyframes, it generates an OOG at each keyframe to model the state of task-relevant objects and the human hand in V . The creation of OOG nodes can reuse the results from the annotation process. For object nodes, the point clouds for node features are obtained by back-projecting the object segmentation with depth data (Line 16). In the RGB-D case, each point node corresponds to sampled keypoints. Their motion features, 3D trajectories, are back-projected from the TAP trajectories using depth data (Line 17). In the RGB case, we have a single point node containing a 6-DOF object trajectory generated by FoundationPose.

Additionally, grip information is required to specify the interaction points with task-relevant objects and the grip status to be mapped to the robot gripper (Line 18). In the RGB-D case, we use a hand-reconstruction model, HaMeR (Pavlakos

et al., 2023), which gives a reconstructed hand mesh that pinpoints the hand locations at each keyframe. The distances between the fingertips of the mesh help determine the grip status, *i.e.*, whether it is open or closed. In the RGB case, since many in-the-wild videos do not show a human hand or significantly occlude the hand, ORION uses grasp heuristics extracted from the estimated object mesh or point cloud, combined with the estimated 6-DOF pose.

With all the node information, ORION establishes the edge connections between nodes in OOGs, representing contact relations. Since all object and hand locations are computed in the camera frame while the camera extrinsic of V is unknown, there is ambiguity when deciding the spatial relations between objects. We exploit the assumption of tabletop manipulation, where a plane is always present with its normal direction aligned with the z-axis of the world coordinate

Algorithm 1 Manipulation Plan Generation from Video V **Require:** RGB frames $r_{1...N}$ from demonstration video V **Optional:** Depth frames $d_{1...N}$ from demonstration video V **Ensure:** OOG sequence $\{\mathcal{G}_0, \dots, \mathcal{G}_L\}$

```

// 1. Track task-relevant objects
1:  $obj\_list \leftarrow \text{VLM}(r)$ 
2:  $masks_1 \leftarrow \text{GroundedSAM}(r_1, obj\_list)$ 
3:  $masks_{2...N} \leftarrow \text{Cutie.track}(masks_1, r_{2...N})$ 
4: if  $d \neq \emptyset$  then ▷ Depth available
5:    $keypoints \leftarrow \text{sample\_points}(masks_1)$ 
6:    $trajectories \leftarrow \text{CoTracker.track}(keypoints, r)$  ▷ Track object keypoints in  $V$ 
7: else ▷ No depth available
8:    $d \leftarrow \text{estimate\_depth}(r)$ 
9:    $meshes \leftarrow \text{scale\_from\_depth}(\text{InstantMesh}(mask, r), d)$  ▷ Generate object meshes
10:   $trajectories \leftarrow \text{FoundationPose}(r, d, masks, meshes)$  ▷ Track object poses in  $V$ 
11: end if
// 2. Discover keyframes
12:  $keyframes \leftarrow \text{changepoint\_detect}(trajectories)$ 
// 3. Generate OOGs
13:  $T_{table} = \text{estimate\_plane}(d_1)$ 
14: for each keyframe  $l$  in keyframes do
15:    $\mathcal{G}_l \leftarrow \text{empty\_graph}()$ 
16:    $\text{add\_object\_nodes}(\mathcal{G}_l, masks_l, d_l, meshes \text{ if } meshes \neq \emptyset)$ 
17:    $\text{add\_point\_nodes}(\mathcal{G}_l, trajectories_l)$ 
18:    $\text{add\_grasp\_node}(\mathcal{G}_l, d_l, meshes \text{ if } meshes \neq \emptyset)$ 
19:    $G_l \leftarrow \text{transform}(T, \mathcal{G}_l)$  ▷ Align OOG features with plane
20: end for
21: return  $\{\mathcal{G}_l\}_{l=0}^L$ 

```

system (Lines 13, 19). ORION estimates the transformation matrix of the table plane and transforms all the point cloud features in OOGs to align with the x-y plane of the world coordinate (Full details appear in Appendix A.2). Then, the contact relations in each state can be determined based on the spatial relations and the computed distances between point clouds. The relations allow ORION to match the test-time observations with a keyframe state from the plan and subsequently decide which object to manipulate (see Section 4.3). In the end, ORION generates a complete OOG for each discovered keyframe.

4.3 Robot Policy To Synthesize Actions

Given a manipulation plan, ORION constructs a policy that synthesizes actions, detailed in Figure 3 and Algorithm 2. The manipulation policy is derived based on the aforementioned learning objective to achieve object motion similarities. The action synthesis comprises three major steps: identify a keyframe from the plan that matches the current observation, predict object motions, and use the predictions to optimize the robot actions for the robot controller to execute. The policy repeats these three steps until a task is completed or fails. Task completion conditions are detailed in Appendix C.

Retrieving OOGs from the plan. ORION first identifies the keyframe and retrieves OOGs to help decide what next actions to take (Lines 2-4). At test-time, ORION local-

izes objects in the new observations and estimates contact relations using the same vision pipeline as described in Section 4.2. Then, ORION retrieves the OOG that has the same set of relations as the current state, allowing us to identify a pair $(\mathcal{G}_l, \mathcal{G}_{l+1})$, where $\mathcal{G}_l$ is the retrieved graph and $\mathcal{G}_{l+1}$ the graph of the next keyframe. This pair of graphs provides sufficient information to decide which object to manipulate next, termed the *target object*, and we denote its point cloud/pose at keyframe l as $\hat{o}_{\text{target}}$, and its object-centric feature trajectories as τ_{target}^V (Lines 5-6). A target object is the one in motion due to manipulation between two keyframes. In the RGB-D case, the target object is identified by computing the average velocity per-object using motion features in G_l . At the same time, another object, called the *reference object*, is involved in changing contact state relations from $\mathcal{G}_l$ to $\mathcal{G}_{l+1}$ and serves as a spatial reference for the target object's movement. In the RGB case, ORION uses the outputs of the classification done by the VLM to determine which objects are target and reference objects. In both cases, ORION use the point cloud of the reference object at the *next keyframe* $l + 1$, as the reference object might have location changes due to object interactions, and using the updated information from the next keyframe gives us an accurate prediction of the trajectories. Once the target and reference objects are determined, we can localize the corresponding objects in the test-time observations, where their point clouds/poses are denoted as o_{target} and o_{ref} , respectively.

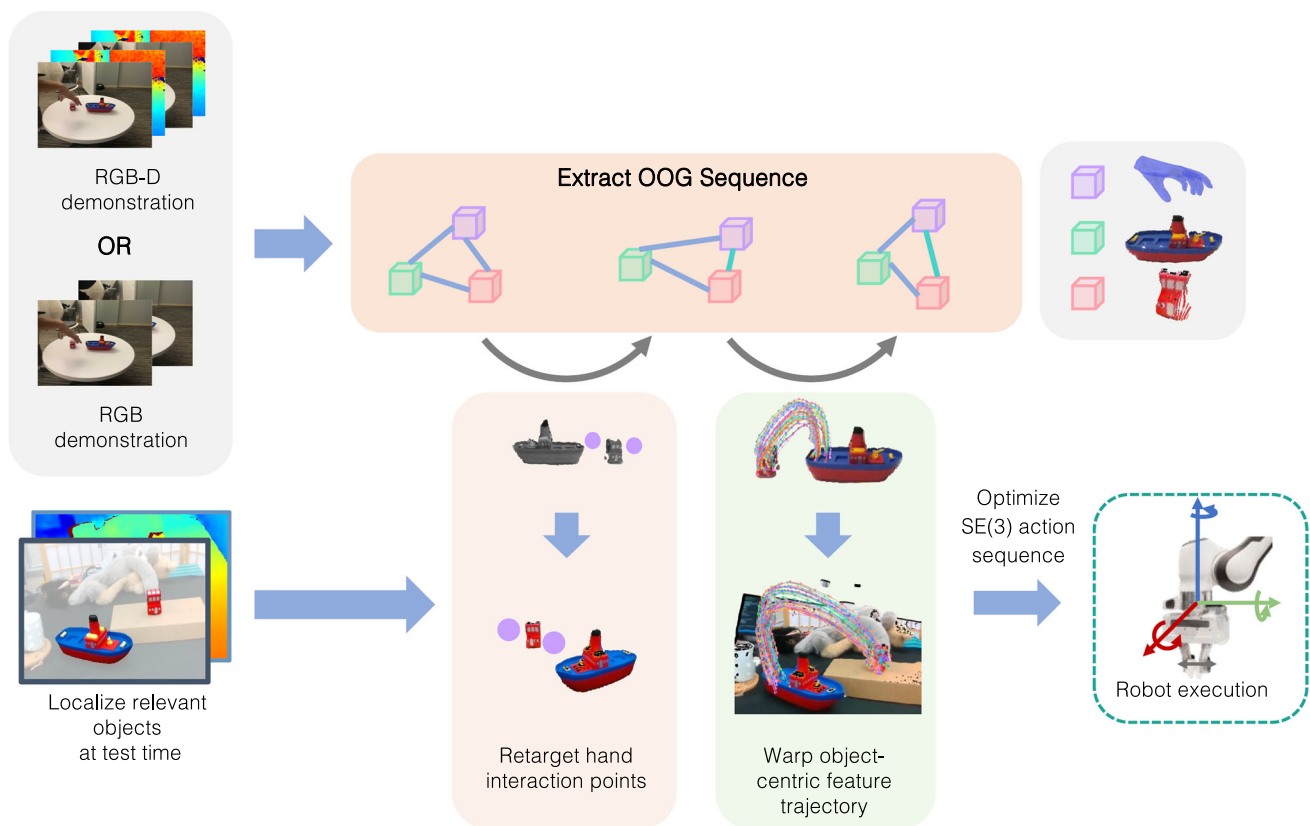


Fig. 3 Overview of the ORION Policy. ORION first localizes task-relevant objects at test time and retrieves the matched OOG from the generated manipulation plan. Then, ORION uses the retrieved OOGs to predict the object motions by warping the object-centric trajec-

tory from the video to match the test-time observation. The predicted trajectories are then used to optimize the SE(3) action sequence of the robot end effector, which is subsequently used to command the robot

Predicting object motions. Given the target and reference objects from keyframes l and $l + 1$, ORION predicts the motion of the target object in the current state by warping the object-centric feature trajectories estimated from V (Lines 7–8). To warp the trajectories, ORION first identifies the initial and goal locations of keypoints in the new configuration by leveraging information given by the OOG pair. In the RGB-D case, ORION uses global registration of point clouds (Choi et al., 2015) to align $\hat{o}_{\text{target}}$ with o_{target} and $\hat{o}_{\text{ref}}$ with o_{ref} , giving two transformations to compute the new starting and goal positions of target object keypoints conditioned on where the reference object is. In the RGB case, since we have the 6-DOF poses of $\hat{o}_{\text{target}}$, o_{target} , $\hat{o}_{\text{ref}}$, and o_{ref} , ORION directly finds the relative transformations required to align the respective pairs.

Now, given new starting and goal positions of the target objects, ORION must warp the original object-centric feature trajectory to fit these new start and end conditions. In the RGB-D case, ORION is warping a 3D point trajectory, which we call *translation warping* (full details in Algorithm 3). ORION first normalizes τ_{target}^V with its starting and goal

locations, obtaining $\hat{\tau}_{\text{target}}$. $\hat{\tau}_{\text{target}}$ only contains the directional and curvature patterns that are independent of the absolute location of the initial and the goal keypoints. Then, $\hat{\tau}_{\text{target}}$ is scaled back to the workspace coordinate frame using the new starting and goal locations, resulting in new keypoint trajectories of the target object $\tau_{\text{target}}^{Ro}$.

In the RGB case, since we have a 6-DOF pose trajectory, ORION also needs to do *rotation warping* in addition to translation warping (full details in Algorithm 4). In rotation warping, ORION first computes the alignment rotation A that maps the demonstration’s reference-to-target object direction to the test-time reference-to-target object direction, and left-multiplies A on every rotation in τ_{target}^V . ORION then normalizes the resulting sequence about its first frame—so it begins at the identity rotation—yielding a “relative rotation” trajectory.

Next, ORION scales this normalized sequence back to the workspace by defining two boundary rotations: one that maps the normalized start to the real initial pose, and one that maps the normalized end to the real goal pose. ORION uses SLERP between these two rotations to get a smooth

Algorithm 2 Robot Policy to Synthesize Actions

Require: Manipulation plan $\mathcal{G} = \{\mathcal{G}_0, \dots, \mathcal{G}_L\}$, Demonstration video V
Ensure: Sequence of robot end-effector SE(3) actions, executed on robot

```

1: while not task_completed and not task_failed do
  // 1. Retrieve matching OOGs from plan
2:   $r^{Ro}, d^{Ro} \leftarrow \text{get\_robot\_obs}()$ 
3:   $\mathcal{G}^{Ro} \leftarrow \text{generate\_OOG}(r^{Ro}, d^{Ro})$  ▷ Same vision pipeline as in 4.2
4:   $(\mathcal{G}_l, \mathcal{G}_{l+1}) \leftarrow \text{find\_matching\_OOG}(\mathcal{G}^{Ro}, \mathcal{G})$ 
5:   $\hat{o}_{target}, \hat{o}_{ref}, \tau_{target}^V \leftarrow \text{extract\_objs\_demo}(\mathcal{G}_l)$ 
6:   $o_{target}, o_{ref} \leftarrow \text{extract\_objs\_test}(\hat{o}_{target}, \hat{o}_{ref}, \mathcal{G}_l, \mathcal{G}_{l+1}, \mathcal{G}^{Ro})$ 
  // 2. Predict object motions
7:   $s_{new}, e_{new} = \text{find\_rollout\_start\_end}(\hat{o}_{target}, o_{target}, \hat{o}_{ref}, o_{ref})$ 
8:   $\tau_{target}^{Ro} \leftarrow \text{warp\_trajectories}(\tau_{target}^V, s_{new}, e_{new})$  ▷ Details in Algorithm 3 and 4
  // 3. Optimize robot actions
9:   $\{T_i\}_{i=0}^{t_{l+1}-t_l} \leftarrow \arg \min_{T_0, T_1, \dots, T_{l+1}-t_l} \sum_{i=0}^{t_{l+1}-t_l} \|\tau_{target}^{Ro}(i+1) - T_i \tau_{target}^{Ro}(i)\|_2^2$ 
10:   $joint\_seq \leftarrow \text{IK}(\{T_i\})$ 
11:   $joint\_seq \leftarrow \text{add\_grip\_info}(joint\_seq, \mathcal{G}_l.vh)$ 
12:   $\text{robot\_execution}(joint\_seq)$ 
13: end while

```

transition $\{I_i\}$, and post-multiplies the normalized rotations by I_i , at each timestep. This set of operations guarantees the warped rotation begins and ends exactly at the real-scene boundary conditions, while still containing the same rotation trajectory “shape” from the original τ_{target}^V .

Optimizing robot actions. Once we obtain τ_{target}^{Ro} , we optimize for a sequence of SE(3) transformations that guide the robot end-effector to move. The SE(3) transformations are optimized to align the keypoint locations from previous frames to the next frames along the predicted trajectories:

$$\arg \min_{T_0, T_1, \dots, T_{t_{l+1}-t_l}} \sum_{i=0}^{t_{l+1}-t_l} \|\tau_{target}^{Ro}(i+1) - T_i \tau_{target}^{Ro}(i)\|_2^2 \quad (1)$$

where $\tau_{target}^{Ro}(i)$ ($0 \leq i \leq t_{l+1} - t_l$) represents the keypoint locations at timestep i along the trajectory (Line 9) and t_l represents the time index for the keyframe l . This optimization process naturally allows generalizations over spatial variations, as the action sequence always conditions on a new location instead of overfitting to absolute locations. In the RGB case, however, because ORION already has the warped 6-DOF pose trajectory of the object, we no longer need to solve Equation 1 and can find T_i in the 6-DOF object pose trajectory.

To further specify where the gripper should interact with the object and whether it should be open or closed, ORION augments the resulting SE(3) sequence with the interaction information stored in the grip node h (Line 11). We implement a combination of inverse kinematics (IK) and joint impedance control to achieve precise and compliant execution (Lines 10, 12).

The resulting ORION policy is robust to visual variations due to the use of open-world vision models. It also general-

izes to different spatial locations due to our choice of representing object locations in object-centric frames and the optimization process that is not constrained to specific positions.

5 Experiments

In this section, we report on experiments to answer the following questions regarding the effectiveness of ORION and the importance of its design choices:

1. Is ORION effective in constructing manipulation policies from single human videos in the open-world setting?
2. To what extent do the object-centric abstractions improve the policy performance?
3. How critical is it to model the object motions with keypoints and the TAP formulation?
4. How consistent is ORION’s performance on videos taken in varied conditions?
5. How does ORION compare to contemporary baselines that rely on different test-time assumptions?
6. Can ORION scale to multi-step, long-horizon manipulation tasks?
7. Can ORION effectively construct manipulation policies from RGB-only demonstration videos in the absence of depth information?

5.1 Experiment Setup

Task descriptions. We design the following ten tasks to evaluate the policy performance:

1. **Mug-on-coaster:** placing a mug on the coaster.

2. *Simple-boat-assembly*: putting a small red block on a toy boat.
3. *Chips-on-plate*: placing a bag of chips on the plate.
4. *Succulents-in-llama-vase*: inserting succulents into the llama vase.
5. *Pour-juice*: pouring juice into a container.
6. *Peas-on-plate*: putting a can of peas on a plate.
7. *Cheese-on-plate*: putting a block of cheese on a plate.
8. *Rearrange-mug-box*: placing a mug on a coaster and placing a cream cheese box on a plate consecutively.
9. *Complex-boat-assembly*: placing both a small red block and a chimney-like part on top of a boat.
10. *Prepare-breakfast*: placing a mug on a coaster and putting a food box and can on the plate.

The first seven are “short-horizon” tasks, and the last three are “long-horizon” tasks. In the context of this paper, “short-horizon” refers to tasks that only require one contact relation between two objects, while “long-horizon” refers to those that require more than one contact relation. Finally, we note that *Mug-on-coaster* and *Succulents-in-llama-vase* have both RGB and RGB-D demonstration videos, while *Pour-juice*, *Peas-on-plate*, and *Cheese-on-plate* only have RGB demonstration videos.

Detailed success conditions of all tasks are described in Appendix C.

Experimental setup. We design experiments to fully test the efficacy of our method by providing the robot with videos captured in everyday scenarios, which naturally encompass visual backgrounds and camera setups that are different from those of the robot. Specifically, we record an RGB-D video of a person performing seven tasks in everyday scenarios, such as an office or a kitchen. We use an iPad for recording, which comes with a TrueDepth Camera, and we fix it on a camera stand. We also evaluate our method on five RGB-only videos which are obtained from YouTube, Veo 2 (Deepmind, 2024), and the previously stated RGB-D videos with the depth channel removed. These additional sources of demonstration provide further material for testing our method’s robustness to varied demonstration environments. All videos can be found on the project website. During test time, the robot receives visual data through a single RGB-D camera, Intel RealSense D435, and performs manipulation in its workstation to evaluate policies. We use the 7-DoF Franka Emika Panda robot for all the experiments.

Evaluation protocol. We design our experiments to test performance along four axes of generalization: 1) changing visual backgrounds, 2) different camera angles, 3) different object instances from the same categories, and 4) varied spatial layouts of the task-relevant objects. As described in

the experimental setup, the demonstration videos naturally include varying visual backgrounds and camera perspectives that are significantly different from the robot workspace, allowing us to test axes 1) and 2). We further test generalization along axis 2) through Question 4) in Section 5.2, where we evaluate ORION’s performance on one task using multiple demonstration videos with different backgrounds and camera angles. We test axis 3) by evaluating on unseen object instances within the same class in the tasks *Mug-on-coaster*, *Chips-on-plate*, *Pour-juice*, *Peas-on-plate*, and *Cheese-on-plate*. All tasks test axis 4), as each trial has a different spatial layout. The policy performance of a task is the average success rate over 15 real-world trials. Aside from the success rates, we also group the failed executions into three types: *Missed tracking* of objects due to failure of the vision models, *Missed grasping* of objects during execution, and *Unsatisfied contacts* where the target object configurations are not achieved for reasons other than the previous two failure types. Visualizations of failure cases can be found on the project website.

Baselines. To understand the model capacity and validate our design choices, we compare ORION with baselines. Since, no prior work exactly ¹ matches our setting without additional test-time assumptions, we adopt the most important components from prior works and treat them as baselines to our model. Additionally, we compare against a contemporary end-to-end method, with and without its assumptions. Specifically, we implement three baselines: 1) *HAND- MOTION- IMITATION* (Bharadhwaj et al., 2023; Wang et al., 2023) is a baseline that predicts robot actions by learning from the hand trajectories. The rest of the parts remain the same as ORION. We use this baseline to show whether it is critical to compute actions centered around objects. 2) *DENSE- CORRESPONDENCE* (Heppert et al., 2024; Ko et al., 2023) is a baseline that replaces the TAP model in ORION with a dense correspondence model, optical flow. This baseline is used to evaluate whether our choice of the TAP model is a better design. For this ablative study, we conduct experiments on *Mug-on-coaster* and *Simple-boat-assembly* to validate our model design, covering the distribution of common daily objects and assembly manipulation that requires precise control. 3) *RIGVID* (Patel et al., 2025) is a recent method that generates a task video given an initial observation and language instruction, and then estimates object poses (using a high-quality scanned object mesh) from the generated video to derive a robot execution trajectory. We use RIGVid to evaluate how a contemporary video-generation-based pipeline performs in our setting, and to test how its performance changes when its test-time assumptions are relaxed and parts of its pipeline are replaced with ORION’s. Concretely, we evaluate RIGVid

¹ See Section 2 for detailed discussion on related work

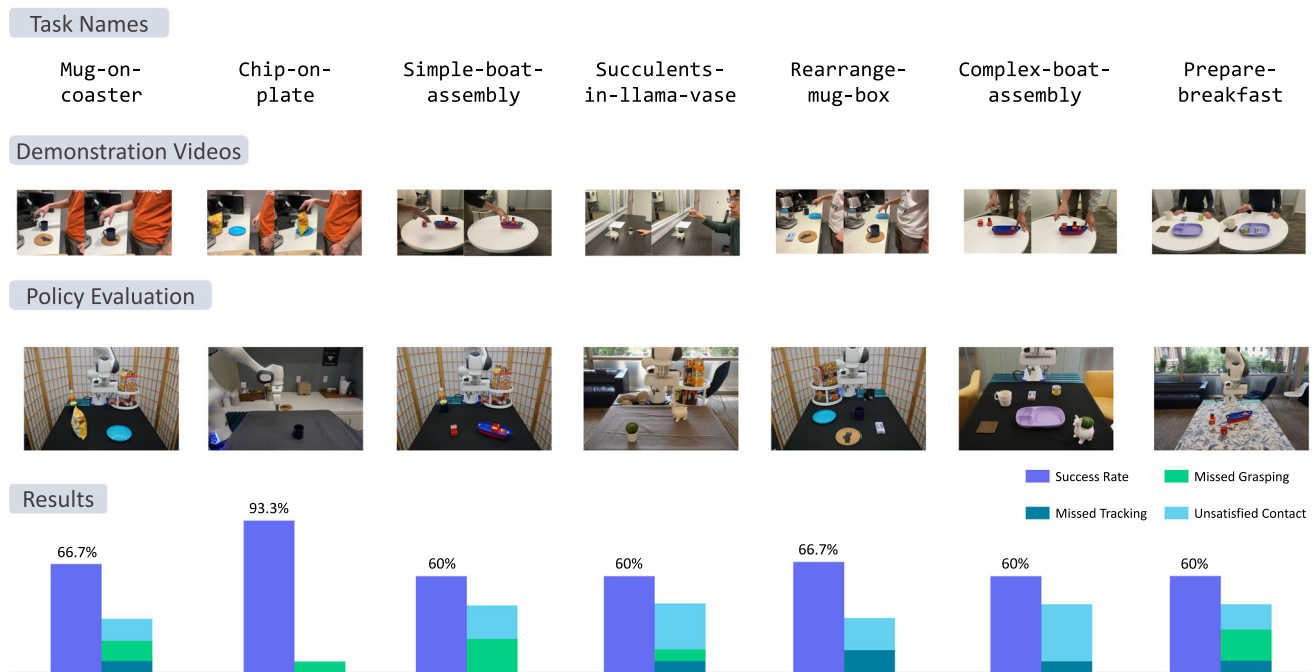


Fig. 4 The upper part of the figure illustrates the following items: the initial and final frames of human videos for every task, the list of word descriptions provided along with the video, and the example images of

initial states for policy evaluation. The lower part of the figure shows the overall evaluation of ORION over all seven tasks, including the success rates and the quantification of failed trials, separated by failure mode

on Cheese-on-plate under three conditions: (i) using an estimated mesh from InstantMesh (assumption-lifted setting), (ii) using the ground-truth scanned object mesh (matched-to-RIGVid setting), and (iii) using the ground-truth mesh while replacing RIGVid's pose estimation with ORION's pose estimation pipeline (isolating whether failures stem from pose estimation versus test-time video generation). Following RIGVid's procedure, we generate up to five demonstration videos and use VLM-based filtering to pick the best one. Because RIGVid relies on commercial video generation models with non-trivial per-rollout cost, we evaluate RIGVid and re-evaluate ORION in Robosuite (Zhu et al., 2020) for controlled, repeatable experiments and environments.

5.2 Experimental Results

Question 1 We show the successful deployment of the ORION policies across 7 tasks with 15 real-world trials each in Figure 4, while no other methods are designed to operate in our setting without additional test-time assumptions. ORION yields an average success rate of 66.7% (95% Wilson CI - [57.2%, 75.0%]), validating our model design in learning from a single human video in the open-world setting.

Question 2 In Figure 5(a), we show the comparison results against the baseline, HAND-MOTION-IMITATION, which

yields low success rates in both tasks. Concretely, HAND-MOTION-IMITATION typically succeeds in trials where the initial spatial layouts are similar to the one in V . Its major failure mode is not being able to reach the target object configuration, e.g., misplacing the mug on the table while not achieving contact with the coaster. These results imply that learning from human hand motion from V results in poor generalization abilities of policies, supporting the design choice of ORION which focuses on the object-centric information.

Question 3 We compare the performance between ORION and the baseline, DENSE-CORRESPONDENCE, shown in Figure 5(a). We observe that the optical flow baseline performs drastically worse on Simple-boat-assembly than on Mug-on-coaster. With our further investigation, we find that the optical flow baseline discovers keyframes in the middle of smooth transitions as opposed to changes in object contact relations, resulting in a manipulation plan that computes completely wrong actions. This finding further supports our choice of using TAP keypoints to discover the keyframes.

Question 4 To evaluate ORION's performance on demonstration videos with varied camera angles and backgrounds, we conduct controlled experiments using the task Mug-on-coaster. Specifically, we record two additional videos of the same task in very different visual conditions and spatial layouts (see details in Appendix D) and construct a policy from each video. Then, we compare the two poli-

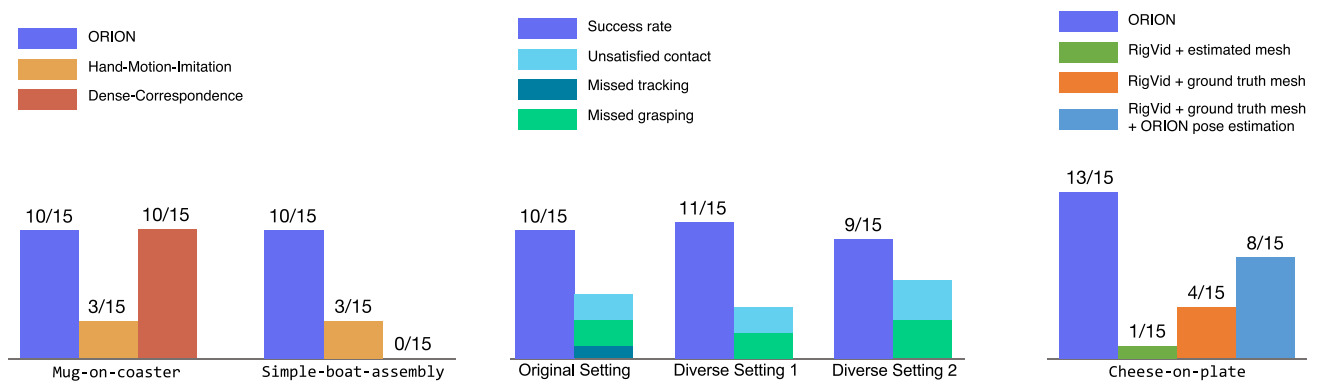


Fig. 5 (a) Experimental comparison between ORION and two prior-work baselines, namely HAND-MOTION-IMITATION and DENSE-CORRESPONDENCE. (b) Ablation study on using different videos of the same task. We select the task Mug-on-coaster for conducting this ablation. We display the number of successful trials out of 15 total trials

cies against the original one and test them using the same set of evaluation conditions. As shown in Figure 5(b), we do not detect a statistically significant difference in the performance ($\chi^2(2) = 0.6$, $p \approx 0.74$), demonstrating that ORION is robust to videos taken under very different visual conditions.

Question 5 We compare ORION and the baseline RIGVid (Patel et al., 2025), shown in Figure 5. In simulation, we re-evaluate ORION and evaluate RIGVid under three conditions: (i) estimated mesh from InstantMesh, (ii) ground-truth scanned mesh, and (iii) ground-truth mesh with RIGVid's pose estimation replaced by ORION's pose estimation pipeline.

We first observe that ORION performs similarly on the Cheese-on-plate task in the new simulation environment, compared to its real-world performance (See Question 7, Figure 6). However, when using an estimated mesh, RIGVid exhibits substantially lower success, consistent with its reliance on a high-quality scanned mesh for downstream pose tracking. Providing the ground-truth mesh improves performance, but two primary failure modes remain.

1) The generated video can produce undesired object motion. When the target object starts near the image boundary, it may move partially or fully out of frame in the generated sequence, causing pose tracking to fail. This reflects a practical limitation of relying on test-time video generation, since rollouts do not guarantee that objects are centered or remain in frame. However, video generation models can still be used at demonstration time, as we can filter or prompt for initial frames that show the objects closer to the center of the image.

2) Pose tracking degrades due to depth inconsistency. RIGVid uses monocular depth estimation (Ke et al., 2025)

on the bar plots for each setting. Figure 7 in Appendix D visualizes the different settings in this experiment. (c) Experimental comparison between ORION and RIGVid with varying assumptions, evaluated in Robosuite

as a replacement for ground-truth depth input to FoundationPose. We observe inaccurate pose estimates, particularly during object motion. We observed a similar issue when developing ORION, and address it by modifying FoundationPose to search over a range of depths (details in Appendix A.2) rather than relying on a single per-frame depth estimate. Replacing RIGVid's pose estimation module with ORION's improves success, but failures due to unsuitable generated videos persist, indicating that video-generation controllability at test-time remains a limiting factor.

Question 6 We also show that ORION is effective in scaling to long-horizon tasks. This conclusion is supported by the performance among the pairs of Mug-on-coaster versus Rearrange-mug-box, and Simple-boat-assembly versus Complex-boat-assembly. In these two pairs, both the short-horizon tasks are subgoals of their long-horizon counterparts, yet we do not see any performance drop between the two. This result indicates that ORION excels at scaling to long-horizon tasks without a significant drop in policy performance.

Question 7 We observe a similar successful deployment of ORION's policies when demonstration videos only contain RGB information without depth, as shown in Figure 6. Pour-juice is retrieved from an in-the-wild YouTube video, Peas-on-plate and Mug-on-coaster are generated from Veo 2 (Deepmind, 2024), and Succulents-in-llama-vase and Cheese-on-plate are from the same recordings used for RGB-D experiments but with the depth channel removed. We observe an average success rate of 85.3%, showing that our method can perform well even without depth information. In addition, we see that our method is able to

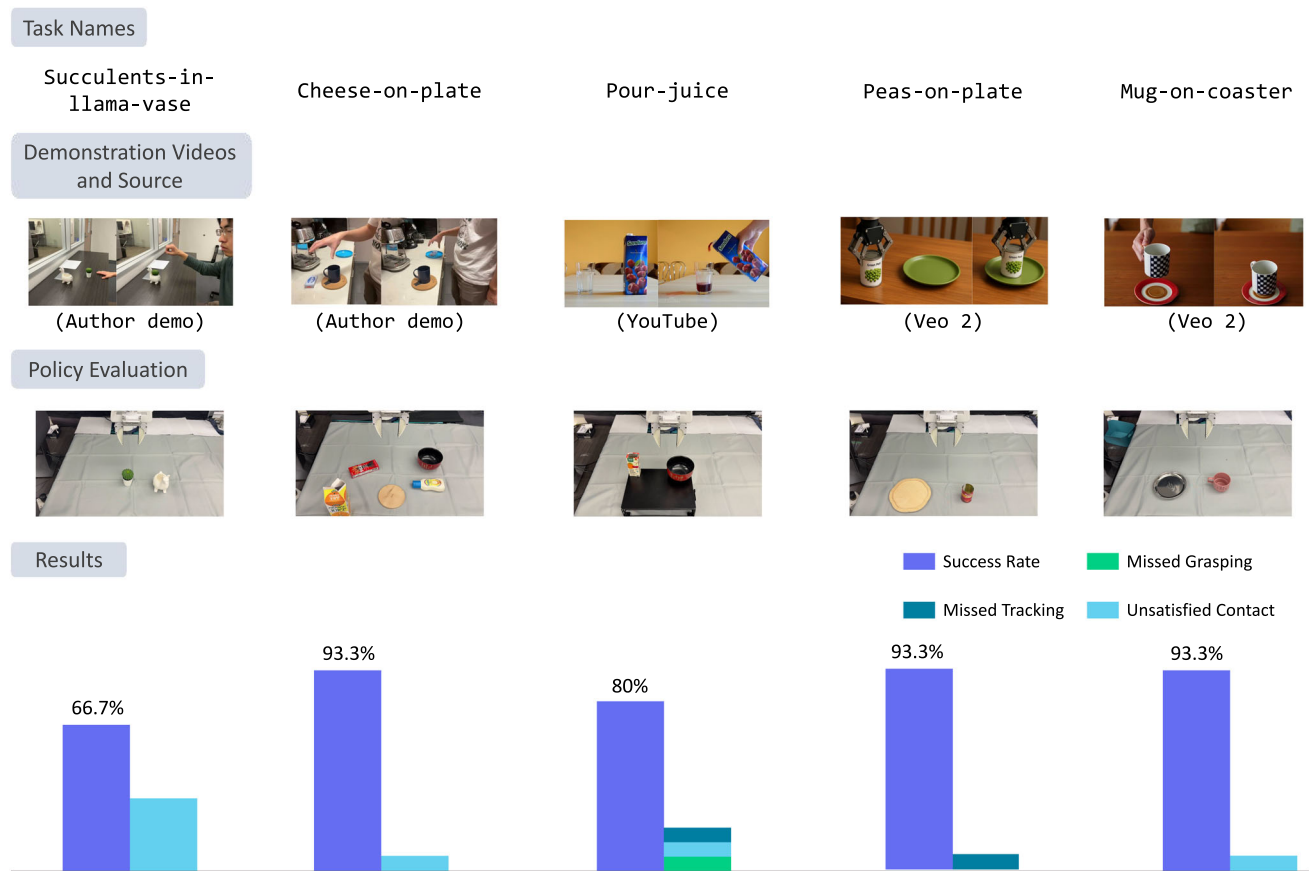


Fig. 6 This figure shows results in the same format as Figure 4, with the difference that demonstration videos do not contain depth information and that videos come from different sources. Note that the

Mug-on-coaster task uses a different demonstration video to showcase compatibility with different object instances, camera angles, and video sources

handle demonstration videos from different sources (from controlled environments, Internet videos, and generated models) and different camera angles, along with test-time object instances and camera angles that differ from the demonstration. These results support our conclusions for Question 4) in the RGB-only setting. To directly compare the RGB-only and RGB-D pipelines, we inspect the results of the *Succulents-in-llama-vase* task, on which both pipelines were evaluated. Interestingly, we see that the RGB-only pipeline performs comparably and even slightly better than the RGB-D pipeline on the same task. We note that certain RGB-only architectural changes, such as the shift from 3D keypoint trajectories to 6-DOF pose trajectories, allow the model to trade noise in global registration and end-effector trajectory optimization for noise in object pose estimation and mesh generation models. We observe that the stronger geometric priors embedded in these models allow our RGB-only pipeline to perform competitively with our RGB-D pipeline, compensating for the lack of depth with minimal added noise.

6 Conclusions

In this paper, we investigate the problem of learning robot manipulation from a single human video in the open-world setting, where a robot must learn to manipulate novel objects using one video demonstration. To tackle this problem, we introduce ORION, an algorithm built on object-centric priors. Our results show that given a single human video, with or without depth, ORION is able to construct a policy that generalizes over the following four dimensions: visual backgrounds, camera angles, spatial layouts, and the presence of new object instances.

Limitations: We consider task goals to be described by contact states, thereby naturally avoiding the ambiguities introduced when considering spatial relations, such as placing items next to an object or avoiding collision with other

objects or the environment. Inferring human intentions while clearing the inherent ambiguities in videos or incorporating motion planning is an interesting future direction to explore. In addition, we do not model all the physical interactions occurring during the task, such as contact dynamics or force regulation. An interesting future direction is to integrate richer physical reconstruction models into the object-centric plan generation, extending the ORION framework to more complex tasks.

We have also assumed that demonstration videos must be captured with a stationary camera. In reality, most videos in everyday scenarios contain a moving camera. Thus, a promising future direction is to investigate how to build a model that can reconstruct dynamic scenes from a moving camera, where the desired model can estimate the geometry of both static and moving objects in the scenes while ensuring the scale of the reconstructed scenes and objects matches the real world. As vision models continue to improve, stronger perceptual back-ends can be integrated into our framework with minimal modification, further boosting success rates and enabling a broader range of tasks.

Acknowledgements We would like to thank Rutav Shah, Jiayuan Mao, Fangchen Liu, Bowen Wen, and Stan Birchfield for helpful discussions. This work has taken place in the Robot Perception and Learning Group (RPL) and Learning Agents Research Group (LARG) at UT Austin. RPL research has been partially supported by the National Science Foundation (FRR2145283, EFRI-2318065) and the Office of Naval Research (N00014-22-1-2204). LARG research is supported in part by the National Science Foundation (FAIN-2019844, NRT-2125858), the Office of Naval Research (N00014-24-1-2550), Army Research Office (W911NF-17-2-0181, W911NF-23-2-0004, W911NF-25-1-0065), DARPA (Cooperative Agreement HR00112520004 on Ad Hoc Teamwork), Lockheed Martin, and Good Systems, a research grand challenge at the University of Texas at Austin. The views and conclusions contained in this document are those of the authors alone. Peter Stone serves as the Chief Scientist of Sony AI and receives financial compensation for that role. The terms of this arrangement have been reviewed and approved by the University of Texas at Austin in accordance with its policy on objectivity in research.

Author Contributions A.L. and Yi.Z. drafted the manuscript text and conducted experiments. P.S. and Yu.Z. provided feedback. All authors reviewed the manuscript.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Competing interests Peter Stone serves as the Chief Scientist of Sony AI and receives financial compensation for that role. The terms of this arrangement have been reviewed and approved by the University of Texas at Austin in accordance with its policy on objectivity in research.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative

Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

Appendix A Additional Technical Details

A.1 Data Structure of an OOG.

For easy reproducibility of the proposed method, we present a table that explains the data structure of an OOG in Table A.1.

A.2 Implementation Details

Changepoint detections. We use changepoint detection to identify changes in the velocity statistics of TAP keypoints. Specifically, we use a kernel-based changepoint detection method and choose radial basis function (Killick et al., 2012). The implementation of this function is directly based on an existing library Ruptures (Truong et al., 2020).

Plane estimation. In Section 4.2, we mentioned using the prior knowledge of tabletop manipulation scenarios and transforming the point clouds by estimating the table plane. Here, we explain how the plane estimation is computed. Concretely, we rely on the plane estimation function from Open3D (Zhou et al., 2018), which gives an equation in the form of $ax + by + cz = d$. From this estimated plane equation, we can infer a normal vector of the estimated table plane, (a, b, c) , in the camera coordinate frame. Then, we align this plane with xy plane in the world coordinate frame, where we compute a transformation matrix that displaces the normal vector (a, b, c) to the normalized vector $(0, 0, 1)$ along the z-axis of the world coordinate frame. This transformation matrix is used to transform point clouds in every frame so that the plane of the table always aligns with the xy plane of the world coordinate.

Object localization at test time. When we localize objects at test time, there could be some false positive segmentation of distracting objects. Such vision failures will prevent the robot policy from successfully executing actions. To exclude such false positive object segmentation, we use Segmentation Correspondence Model (SCM) from GROOT (Zhu et al., 2023), where SCM filters out the false positive segmentation of the objects by computing the affinity scores between masks using DINOv2 features.

Table 1 Data Structure of an OOG. For a given OOG $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, it has $\mathcal{V} = \{\mathcal{G}.vo_i\} \cup \{\mathcal{G}.vh\} \cup \{\mathcal{G}.vp_{ij}\}$, and $\mathcal{E} = \{\mathcal{G}.eo_{ik}\} \cup \{\mathcal{G}.eh_i\} \cup \{\mathcal{G}.ep_{ij}\}$

Node/Edge	Type	Attributes
$\mathcal{G}.vo_i$	Object Node	3D point cloud of an object (+ object mesh if RGB-only).
$\mathcal{G}.vh$	Grasp Node	Grasp locations on object, calculated through hand mesh finger locations / object mesh heuristics.
$\mathcal{G}.vp_{ij}$	Point Node	A trajectory of a TAP keypoint (RGB-D) or 6-DOF object pose (RGB-only) between two keyframes.
$\mathcal{G}.eo_{ik}$	Object-Object Edge	A binary value of contact or not.
$\mathcal{G}.eh_i$	Object-Hand Edge	A binary value of contact or not.
$\mathcal{G}.ep_{ij}$	Object-Point Edge	The presence of an edge represents the belonging relation, and no specific feature is attached.

Global registration. In this paper, we use global registration to compute the transformation between observed object point clouds from videos and those from rollout settings. We implement this part using a RANSAC-based registration function from Open3D (Zhou et al., 2018). Specifically, given two object point clouds, we first compute their features using Fast-Point Feature Histograms (FPFH) (Rusu et al., 2009), and then perform a global RANSAC registration on the FPFH features of the point clouds (Choi et al., 2015).

Implementation of SE(3) optimization. We parameterize each homogeneous matrix T_i into a translation variable and a rotation variable and randomly initialize each variable using the normal distribution. We choose quaternions as the representation for rotation variables, and we normalize the randomly initialized vectors for rotation so that they remain unit quaternions. With such parameterization, we optimize the SE(3) end-effector trajectories $T_0, T_1, \dots, T_{t_{l+1}-t_l}$ over the Objective (1). However, jointly optimizing both translation and rotation from scratch typically results in trivial solutions, where the rotation variables do not change much from the initialization due to the vanishing gradients. To avoid trivial solutions, we implement a two-stage process. In the first stage, we only optimize the rotation variables with 200 gradient steps. Then, the optimization proceeds to the second stage, where we optimize both the rotation and translation variables for another 200 gradient steps. In this case, we prevent the optimization process from getting stuck in trivial solutions for rotation variables. We implement the optimization process using Lietorch (Teed & Deng, 2021).

FoundationPose implementation. The off-the-shelf FoundationPose implementation does not support RGB-only inputs. To adapt it for our use case, we modify the implementation to take in a tuple (l, h) where l and h represent the lower and upper bounds of the distance from the camera to the object, respectively. We estimate this upper and lower bound by applying monocular depth estimation on the object, calculating the median depth over all pixels belonging to the object, and setting $l = med - 0.15$ and $h = med + 0.15$. We then create pose hypotheses centered at regular intervals between l and h . These pose hypotheses are then sent to the

pose refiner, and from here, the implementation follows the original FoundationPose.

Scaling object meshes. The meshes output by InstantMesh are of arbitrary scale. To deal with this, we use the estimated depth from monocular depth estimation models to backproject each pixel within the object mask, creating a point cloud of that object. To remove outliers in the point cloud, we use Open3D's `remove_statistical_outlier` and `cluster_dbscan` methods. Next, we find the axis-aligned bounding box of the point cloud and mesh. We calculate a scale factor $s = \frac{p}{m}$ where p and m are the diagonal lengths of the point-cloud and mesh bounding boxes, respectively. Finally, we uniformly apply s to the generated mesh to get an approximation of the true mesh scale.

ORION RGB vs RGB-D implementation. Below we summarize the implementation-level differences between our RGB-D and RGB-only pipelines. These differences affected how object correspondence and end-effector (EE) trajectories are obtained, and which assumptions are introduced in the pipeline

- **Tracking and transform estimation.** Our RGB-D pipeline tracks 3D keypoint trajectories on objects and uses *global registration* of object point clouds to estimate the correspondence transform between demonstration and rollout scenes:

$$T_{\hat{o} \rightarrow o} = \text{GLOBAL_REGISTRATION}(\hat{o}, o),$$

where $\hat{o}$ and o are object point clouds of the demonstration and rollout objects, respectively.

The RGB-only pipeline directly estimates the full 6-DOF pose of the target object in each frame during the demonstration, along with the initial pose of the target and reference objects during rollout. Thus, we only need to find the desired end pose of the target object during rollout. In this case, o and $\hat{o}$ refers to the object's pose. We find the rollout target end pose with

$$o_{\text{target}}^{\text{end}} = o_{\text{ref}}^{\text{start}} \cdot (\hat{o}_{\text{ref}}^{\text{start}})^{-1} \cdot \hat{o}_{\text{target}}^{\text{start}}$$

Algorithm 3 Translation Warping**Require:** 3D demonstration object trajectory τ , Desired 3D new start and end points s, e .**Ensure:** Warped 3D object trajectory τ^{warped}

```

// Normalize trajectory and calculate rotation with Rodrigues formula
1:  $\tau^{norm} \leftarrow \{\tau_i - \tau_1 \mid \forall i \in \{1, \dots, L\}\}$ 
2:  $v_1, v_2 \leftarrow \tau_L - \tau_1, e - s$ 
// Calculate rotation with Rodrigues formula
3:  $A \leftarrow \text{Rodrigues}(v_1, v_2)$ 
4:  $scale \leftarrow \frac{\|v_2\|}{\|v_1\|}$ 
// Transform back to workspace frame
5:  $\tau^{warped} \leftarrow \{scale(A \cdot \tau^{norm}) + s\}$ 
6:
7: return  $\tau^{warped}$ 

```

Algorithm 4 Rotation Warping**Require:** 6-DOF demonstration object trajectory τ , desired 6-DOF start s and end e . For some 4x4 6-DOF pose matrix X , we refer to the rotation as $Rot(X) \leftarrow X_{1:3,1:3}$ and translation as $t(X) \leftarrow X_{1:3,4}$.**Ensure:** Warped object rotation trajectory R^{warped}

```

1:  $R \leftarrow Rot(\tau)$ 
2:  $v_1, v_2 \leftarrow t(\tau_L) - t(\tau_1), t(e) - t(s)$ 
// Rotate R to align with demonstration direction
3:  $A \leftarrow \text{Rodrigues}(v_1, v_2)$ 
4:  $R' \leftarrow \{A \cdot R_i \mid \forall i \in \{1, \dots, L\}\}$ 
5:  $R^{norm} \leftarrow \{R'_i \cdot (R'_1)^T \mid \forall i\}$ 
// Calculate boundary rotations and apply spherical linear interpolation
6:  $slerp_{start} \leftarrow Rot(s), slerp_{end} \leftarrow (R^{norm})^T \cdot Rot(e)$ 
7:  $I \leftarrow SLERP(slerp_{start}, slerp_{end})$ 
// Combine interpolated rotations with original trajectory
8:  $R^{warped} \leftarrow \{R^{norm}_i \cdot I_i \mid \forall i\}$ 
9: return  $R^{warped}$ 

```

removing the need for an intermediate point-cloud registration step

- **End-effector (EE) pose sequence.** The RGB-D pipeline finds the EE sequence by solving the optimization problem in 1. However, in the RGB-only case, because we already have the pose sequence of the object, we can apply a single transform to all elements of o_{target} to retrieve $\{T\}$
- **Grasp selection.** The RGB-only pipeline chooses grasp locations using object geometry heuristics derived from estimated meshes, rather than attempting to recover demonstrated hand/grasps from video. This is an assumption that simplifies grasp selection but can bias results relative to the RGB-D pipeline, which attempts to infer demonstrated hand information. In addition, the pose of this grasp selection is the transform applied to all elements of o_{target} to retrieve the EE pose trajectory.

Object-Centric Trajectory Warping We provide pseudocode to give more detail on the implementation of translation and rotation warping in Algorithm 3 and Algorithm 4.

Appendix B System Setup

Details of camera observations. As mentioned in Section 5, we use an iPad with a TrueDepth camera for collecting RGB-D human video demonstrations. We use an iOS app, Record3D, that allows us to access the depth images from the TrueDepth camera. We record RGB and depth image frames in sizes 1920×1080 and 640×480 , respectively. To align the RGB images with the depth data, we resize the RGB frames to the size 640×480 . The app also automatically records the camera intrinsics of the iPhone camera so that the back-projection of point clouds is made possible. In the RGB-only case, we use the same videos with the depth channel removed, videos generated from generative models, and YouTube videos.

To stream images at test time, we use an Intel RealSense D435i. In our robot experiments, we use RGB and depth images in the size 640×480 or 1280×720 in varied scenarios, all covered in our evaluations. Evaluating on different image sizes showcases that our method is not tailored to specific camera configurations, supporting the wide applicability of the constructed policy.

Implementation of real robot control. In our evaluation, we reset the robot to a default joint position before object interaction every time. Then we use a reaching primitive for the robot to reach the interaction points. Resetting to the default joint

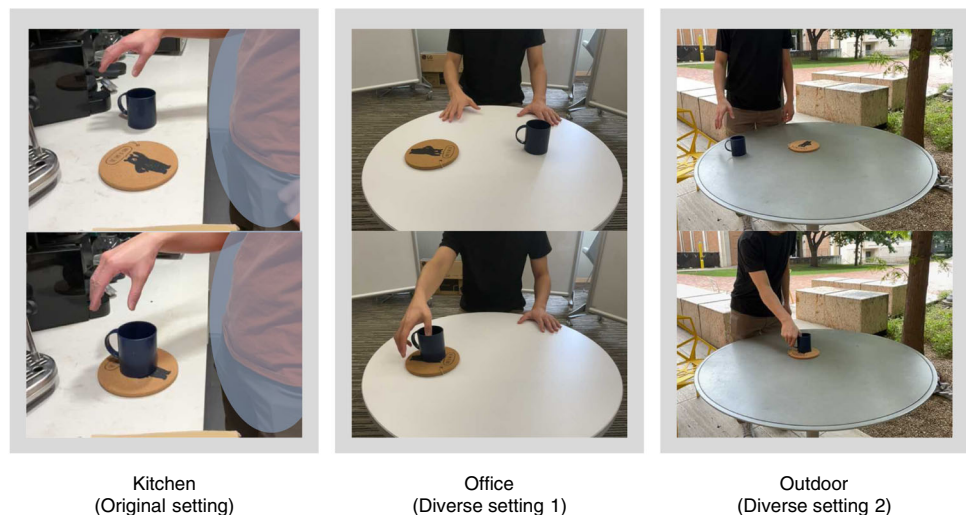
position enables an unoccluded observation of task-relevant objects at the start of each decision-making step. Note that the execution of object interaction does not necessarily require resetting. To command the robot to interact with objects, we convert the optimized SE(3) action sequence to a sequence of joint configurations using inverse kinematics and control the robot using joint impedance control. We use the implementation of Deoxys (Zhu et al., 2022) for the joint impedance controller that operates at 500 Hz. To avoid abrupt motion and make sure the actions are smooth, we further interpolate the joint sequence from the result of inverse kinematics. Specifically, we choose the interpolation so that the maximal displacement for each joint does not exceed 0.5 radian between two adjacent waypoints.

Appendix C Success conditions of tasks

We describe the success conditions for each of the tasks in detail:

- **Mug-on-coaster:** A mug is placed upright on the coaster.
- **Simple-boat-assembly:** A red block is placed in the slot closest to the back of the boat. The block needs to be upright in the slot.
- **Chips-on-plate:** A bag of chips is placed on the plate, and the bag does not touch the table.
- **Succulents-in-llama-vase:** A pot of succulents is inserted into a white vase in the shape of a llama.
- **Rearrange-mug-box:** The mug is placed upright on the coaster, and the cream cheese box is placed on the plate.
- **Complex-boat-assembly:** The chimney-like part is placed in the slot closest to the front of the boat. The red block is placed in the slot closest to the back of the boat. Both blocks need to be upright in the slots.

Fig. 7 This figure visualizes the initial and final frames of the three videos of the same task Mug-on-coaster



- **Prepare-breakfast:** The mug is placed on top of a coaster, the cream cheese box is placed in the large area of the plate, and the food can is placed on the small area as shown in the video demonstration.
- **Peas-on-plate:** The peas can is placed upright in the large area of the plate.
- **Pour-juice:** The juice box is tilted 45 degrees in the direction of its opening, such that the opening ends above a container.
- **Cheese-on-plate:** The cream cheese box is placed in the large area of the plate.

In practice, we record the success and failure of a rollout as follows: If the program in ORION policy returns true when matching the observed state with the final OOG from a plan, we mark a trial as success as long as we observe that the object state indeed satisfies the success condition of a task as described above. Otherwise, if the robot generates dangerous actions (bumping into the table) or does not achieve the desired subgoal after executing the computed trajectory, we consider the rollout as a failure and we manually record the failure.

Appendix D Additional Details on Experiments

Diverse video recordings used in the ablation study. Figure 7 shows the three videos taken in very different scenarios: kitchen, office, and outdoor. The video taken in the kitchen scenario is used in the major quantitative evaluation, termed “Original setting”. The other two settings are termed “Diverse setting 1” and “Diverse setting 2.” We conduct an ablation study where we compare policies imitated from these three videos, which inherently involve varied visual scenes and camera perspectives. The result of the ablation study is shown in Figure 5.

Appendix E Prompts sent to VLM

Identify task-relevant objects during rollout. This prompt is used to identify which objects in the scene correspond to the task-relevant objects in the demonstration by comparing the object similarity.

Prompt: You are given a list of object names (called `obj_names`): `{obj_names}`, a list of object types (called `obj_types`): `{obj_types}`, and images with white text at the bottom. One image is the current observation in real life, which is labeled accordingly. Another image is the observation from the demonstration video, which annotates the objects in the video according to `obj_names`. The rest of the images, which are labeled as “Demo Video Frame x”, show how the `obj_names` move in the demonstration video. `obj_types[i]` describes what type of object `obj_names[i]` is. You can see the corresponding object in the image titled “Demo Observation Image”. If the object type is “manipulate”, that means a person manipulated the object with their hands. If the object type is “reference”, that means the object was used as a reference for the manipulated object. For example, if a person shakes salt from a salt shaker into a bowl, the salt shaker is the manipulated object and the bowl is the reference object, since the person directly manipulated the salt shaker, and the bowl stayed still and received the salt from the salt shaker. For each object name in `obj_names`, return an object name (can be the same name) that best describes the closest related object in the image titled “Real Observation Image”. If the closest related object in the Real Observation Image is exactly the same object as the one in the Demo Observation Image, you can return the same name. Make sure that `obj_names[i]` corresponds to `similar_obj_names[i]`, and `similar_obj_names[i]` is of type `obj_types[i]`. Finally, please make sure the `similar_obj_names[i]` is described thoroughly, including color or shape. In descending order of priority, `similar_obj_names[i]` should have a similar shape, object type, orientation, or function to `obj_names[i]`. Make sure each description is descriptive, but concise (5 words or fewer).

A description of the video from which the demonstration frames were taken is “demo_video_description”

Your output format is:

```
{{
"similar_obj_names": [NAME_OF_SIMILAR_OBJ_1,
                      NAME_OF_SIMILAR_OBJ_2, ...]
"explanation": "Here, you will explain your reasoning for
               choosing similar_obj_names[i] as the
               similar object to obj_names[i]"
}}
```

Example:

```
{{
"similar_obj_names": [red water bottle, checkered plate]
"explanation": "obj_names[0] is 'green thermos', and the
               closest related object in the image is a red water bottle,
               because they are both drinking objects which are both standing
               upright, are both cylindrical, and both around 0.3 meters.
               obj_names[1] is 'purple plate', and the closest related
               object in the image is a checkered plate, which has the
               same shape and function as the purple plate, and can be
               used as a reference object for the red water bottle.
               Taking the whole real-world scene into consideration,
               we can place the red water bottle onto the checkered
               plate, just as the green thermos is placed onto
               the purple plate in the demonstration video."
}}
```

Ensure the response can be parsed by Python `'json.loads'`, e.g.: no trailing commas, no single quotes, etc.

Identify task relevant objects, then classify into target and reference objects. The prompts used here are largely similar to the ones used in OKAMI (Li et al., 2024).

Find task relevant objects prompt: You need to analyze what the human is doing in the images, then tell me: 1. All the objects in front of the scene (mostly on a plane similar to a table). You should ignore the background objects. 2. The objects of interest. They should be a subset of your answer to the first question. They are likely the objects manipulated by the human or objects that interact with other objects. Objects that the human uses (like utensils) to directly manipulate other objects should NOT be included as task relevant objects. Note that there are irrelevant objects in the scene, such as objects that are not interacted with by the human and/or the other objects. You should ignore the irrelevant objects. Make sure to include a color and texture description of the relevant objects. This is especially important if there are many instances of a single type of object. For example, if there are blue, green, and yellow balls, be sure to specify each one individually. Do NOT just say "colorful balls". Finally, if any objects emit liquid, do not include the liquid itself as an object you return. Your output format is:

```
{
  "video_description": "The human is xxx.",
  "all_objects": ["OBJECT1", "OBJECT2", ...],
  "objects_of_interest": ["OBJECT1", "OBJECT3", ...],
}
```

Ensure the response can be parsed by Python 'json.loads', e.g.: no trailing commas, no single quotes, etc.

Example:

```
{
  "video_description": "The human is picking up
a red ball with blue dots and putting it in
a green cup",
  "all_objects": ["blue ball with stripes",
                  "small green ball",
                  "red ball with blue dots",
                  "green cup"],
  "objects_of_interest": ["red ball with blue dots",
                          "green cup"],
}
```

Identify target objects prompt: The following images shows a manipulation motion, where the human is manipulating an object or objects.

Your task is to determine which objects are being manipulated in the images below. You need to choose from the following objects: {task_relevant_object_candidates}.

Tips: A manipulated object is an object that the human is interacting with, such as picking up, moving, or pressing, and it is in contact with the human or an object (like a utensil) that the human is using. If there are multiple objects of the same type, be sure to specify the color, texture, etc. that correspond to the manipulated object

Your output format is:

```
{{
  "manipulate_object_names": [
    "MANIPULATE_OBJECT_DESCRIPTIVE_NAME_1",
    "MANIPULATE_OBJECT_DESCRIPTIVE_NAME_2",
    ...],
}}
```

Example:

```
{{
  "manipulate_object_names": ["red ball with blue dots",
                              "green cup"],
}}
```

Ensure the response can be parsed by Python 'json.loads', e.g.: no trailing commas, no single quotes, etc.

Identify reference objects prompt: The following images shows a manipulation motion, where the human is manipulating the objects in the list - {manipulate_obj_names}.

Please identify the reference object in the image below, which could be an object on which to place an object in the list {manipulate_obj_names}, or an object that something in the list {manipulate_obj_names} is interacting with. You need to first identify whether there is a reference object. If so, you need to output the reference object's name chosen from the following objects: {task_relevant_object_candidates}. Note that there may not necessarily be a reference object. Examples of reference objects are: a coaster, if the motion involves picking up a cup and placing it on a coaster; a bowl, if the motion involves picking up a bottle and pouring it into the bowl.

Finally, make sure your proposed ith reference object corresponds to the given ith manipulated object.

```
{{
  "reference_object_names": ["REFERENCE_OBJECT_NAME_1",
                             "REFERENCE_OBJECT_NAME_2", ...]
                             or ["None"],
  "reasoning": "Here, you will explain your reasoning for
choosing these objects as the reference object,
or for saying that there are no reference objects"
}}
```

Your output format is:

Example:

```
{{
  "reference_object_names": ["white circular plate",
                             "brown coaster"],
  "reasoning": "The white circular plate is a reference
object because the human is placing the red ball on it.
The brown coaster is a reference object because
the human is placing the green cup on it."
}}
```

Ensure the response can be parsed by Python 'json.loads', e.g.: no trailing commas, no single quotes, etc.

References

- Bharadhwaj, H., Gupta, A., Kumar, V. & Tulsiani, S. (2023). Towards generalizable zero-shot manipulation via translating human interaction plans. arXiv preprint [arXiv:2312.00775](https://arxiv.org/abs/2312.00775)
- Bahl, S., Gupta, A. & Pathak, D. (2022). Human-to-robot imitation in the wild. arXiv preprint [arXiv:2207.09450](https://arxiv.org/abs/2207.09450)
- Bharadhwaj, H., Gupta, A., Tulsiani, S. & Kumar, V. (2023). Zero-shot robot manipulation from passive human videos. arXiv preprint [arXiv:2302.02011](https://arxiv.org/abs/2302.02011)
- Chen, A.S., Nair, S. & Finn, C. (2021). Learning generalizable robotic reward functions from "in-the-wild" human videos. arXiv preprint [arXiv:2103.16817](https://arxiv.org/abs/2103.16817)
- Cheng, H.K., Oh, S.W., Price, B., Lee, J.-Y. & Schwing, A. (2023). Putting the object back into video object segmentation. arXiv preprint [arXiv:2310.12982](https://arxiv.org/abs/2310.12982)
- Choi, S., Zhou, Q.-Y. & Koltun, V. (2015). Robust reconstruction of indoor scenes. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 5556–5565.
- Devin, C., Abbeel, P., Darrell, T. & Levine, S. (2018). Deep object-centric representations for generalizable robot learning. In: 2018 IEEE International Conference on Robotics and Automation (ICRA), pp. 7111–7118. IEEE.
- Duan, Y., Andrychowicz, M., Stadie, B., Jonathan Ho, O., Schneider, J., Sutskever, I., Abbeel, P. & Zaremba, W. (2017). One-shot imitation learning. *Advances in neural information processing systems* 30.
- Deepmind, G. (2024). State-of-the-art video and image generation with Veo 2 and Imagen 3. *Google Labs blog*. <https://blog.google/technology/google-labs/video-image-generation-update-december-2024>
- Di Palo, N. & Johns, E. (2022). Learning multi-stage tasks with one demonstration via self-replay. In: Conference on Robot Learning. PMLR. pp. 1180–1189.
- Dalal, M., Pathak, D., & Salakhutdinov, R. R. (2021). Accelerating robotic reinforcement learning via parameterized action primitives. *Advances in Neural Information Processing Systems*, 34, 21847–21859.
- Heppert, N., Argus, M., Welschhold, T., Brox, T. & Valada, A. (2024). Ditto: Demonstration imitation by trajectory transformation. arXiv preprint [arXiv:2403.15203](https://arxiv.org/abs/2403.15203).
- Huang, Y., Conkey, A. & Hermans, T. (2023). Planning for multi-object manipulation with graph neural network relational classifiers. In: 2023 IEEE International Conference on Robotics and Automation (ICRA), pp. 1822–1829. IEEE.
- Haldar, S., Mathur, V., Yarats, D. & Pinto, L. (2023). Watch and match: Supercharging imitation with regularized optimal transport. In: Conference on Robot Learning, pp. 32–43. PMLR.
- Haldar, S., Pari, J., Rai, A. & Pinto, L. (2023). Teach a robot to fish: Versatile imitation from one minute of demonstrations. arXiv preprint [arXiv:2303.01497](https://arxiv.org/abs/2303.01497).
- Joseph, K., Khan, S., Khan, F.S. & Balasubramanian, V.N. (2021). Towards open world object detection. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 5830–5840.
- Johns, E. (2021). Coarse-to-fine imitation learning: Robot manipulation from a single demonstration. In: 2021 IEEE International Conference on Robotics and Automation (ICRA), pp. 4613–4619. IEEE.
- Killick, R., Fearnhead, P., & Eckley, I. A. (2012). Optimal detection of changepoints with a linear computational cost. *Journal of the American Statistical Association*, 107(500), 1590–1598.
- Ko, P.-C., Mao, J., Du, Y., Sun, S.-H. & Tenenbaum, J.B. (2023). Learning to act from actionless videos through dense correspondences. arXiv preprint [arXiv:2310.08576](https://arxiv.org/abs/2310.08576)
- Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., Xiao, T., Whitehead, S., Berg, A.C., Lo, W.-Y., et al. (2023). Segment anything. arXiv preprint [arXiv:2304.02643](https://arxiv.org/abs/2304.02643)
- Ke, B., Narnhofer, D., Huang, S., Ke, L., Peters, T., Fragkiadaki, K., Obukhov, A. & Schindler, K. (2025). Video Depth without Video Models. <https://arxiv.org/abs/2411.19189>.
- Karaev, N., Rocco, I., Graham, B., Neverova, N., Vedaldi, A. & Rupprecht, C. (2023). Cotracker: It is better to track together. arXiv preprint [arXiv:2307.07635](https://arxiv.org/abs/2307.07635).
- Karnan, H., Torabi, F., Warnell, G. & Stone, P. (2022). Adversarial imitation learning from video using a state observer. In: 2022 International Conference on Robotics and Automation (ICRA), pp. 2452–2458. IEEE.
- Kumar, S., Zamora, J., Hansen, N., Jangir, R. & Wang, X. (2023). Graph inverse reinforcement learning from diverse videos. In: Conference on Robot Learning, pp. 55–66. PMLR.
- Liu, Y., Gupta, A., Abbeel, P. & Levine, S. (2018). Imitation from observation: Learning to imitate behaviors from raw video via context translation. In: 2018 IEEE International Conference on Robotics and Automation (ICRA), pp. 1118–1125. IEEE.
- Liu, S., Zeng, Z., Ren, T., Li, F., Zhang, H., Yang, J., Li, C., Yang, J., Su, H., Zhu, J., et al. (2023). Grounding dino: Marrying dino with grounded pre-training for open-set object detection. arXiv preprint [arXiv:2303.05499](https://arxiv.org/abs/2303.05499).
- Li, J., Zhu, Y., Xie, Y., Jiang, Z., Seo, M., Pavlakos, G. & Zhu, Y. (2024). Okami: Teaching humanoid robots manipulation skills through single video imitation. arXiv preprint [arXiv:2410.11792](https://arxiv.org/abs/2410.11792)
- Migimatsu, T., & Bohg, J. (2020). Object-centric task and motion planning in dynamic environments. *IEEE Robotics and Automation Letters*, 5(2), 844–851.
- Mo, K., Guerrero, P., Yi, L., Su, H., Wonka, P., Mitra, N. & Guibas, L.J. (2019). Structurenets: Hierarchical graph networks for 3d shape generation. arXiv preprint [arXiv:1908.00575](https://arxiv.org/abs/1908.00575).
- Ma, Y.J., Sodhani, S., Jayaraman, D., Bastani, O., Kumar, V. & Zhang, A. (2022). Vip: Towards universal visual reward and representation via value-implicit pre-training. arXiv preprint [arXiv:2210.00030](https://arxiv.org/abs/2210.00030)
- Mandlekar, A., Xu, D., Martín-Martín, R., Savarese, S. & Fei-Fei, L. (2020). Learning to generalize across long-horizon tasks from human demonstrations. arXiv preprint [arXiv:2003.06085](https://arxiv.org/abs/2003.06085)
- Nasiriany, S., Liu, H. & Zhu, Y. (2022). Augmenting reinforcement learning with behavior primitives for diverse manipulation tasks. In: 2022 International Conference on Robotics and Automation (ICRA), pp. 7477–7484. IEEE.
- Nair, S., Rajeswaran, A., Kumar, V., Finn, C. & Gupta, A. (2022). R3m: A universal visual representation for robot manipulation. arXiv preprint [arXiv:2203.12601](https://arxiv.org/abs/2203.12601).
- Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El-Nouby, A., et al. (2023). DINOv2: Learning robust visual features without supervision. arXiv preprint [arXiv:2304.07193](https://arxiv.org/abs/2304.07193).
- Park, S., Bharadhwaj, H. & Tulsiani, S. (2025). Demodiffusion: One-shot human imitation using pre-trained diffusion policy. arXiv preprint [arXiv:2506.20668](https://arxiv.org/abs/2506.20668)
- Patel, S., Mohan, S., Mai, H., Jain, U., Lazebnik, S. & Li, Y. (2025). Robotic manipulation by imitating generated videos without physical demonstrations. arXiv preprint [arXiv:2507.00990](https://arxiv.org/abs/2507.00990)
- Pavlakos, G., Shan, D., Radosavovic, I., Kanazawa, A., Fouhey, D. & Malik, J. (2023). Reconstructing hands in 3d with transformers. arXiv preprint [arXiv:2312.05251](https://arxiv.org/abs/2312.05251)
- Pavse, B. S., Torabi, F., Hanna, J., Warnell, G., & Stone, P. (2020). Ridm: Reinforced inverse dynamics modeling for learning from a single observed demonstration. *IEEE Robotics and Automation Letters*, 5(4), 6262–6269.
- Qureshi, A.H., Mousavian, A., Paxton, C., Yip, M.C. & Fox, D. (2021). Nerp: Neural rearrangement planning for unknown objects. arXiv preprint [arXiv:2106.01352](https://arxiv.org/abs/2106.01352).

- Rusu, R.B., Blodow, N. & Beetz, M. (2009). Fast point feature histograms (fpfh) for 3d registration. In: 2009 IEEE International Conference on Robotics and Automation, pp. 3212–3217. IEEE.
- Smith, L., Dhawan, N., Zhang, M., Abbeel, P. & Levine, S. (2019). Avid: Learning multi-stage tasks via pixel-level translation of human videos. arXiv preprint [arXiv:1912.04443](https://arxiv.org/abs/1912.04443).
- Sharma, P., Pathak, D. & Gupta, A. (2019). Third-person visual imitation learning via decoupled hierarchical controller. *Advances in Neural Information Processing Systems* 32.
- Shi, J., Qian, J., Ma, Y.J. & Jayaraman, D. (2024). Plug-and-play object-centric representations from “what” and “where” foundation models.
- Stone, A., Xiao, T., Lu, Y., Gopalakrishnan, K., Lee, K.-H., Vuong, Q., Wohlhart, P., Zitkovich, B., Xia, F., Finn, C., et al. (2023). Open-world object manipulation using pre-trained vision-language models. arXiv preprint [arXiv:2303.00905](https://arxiv.org/abs/2303.00905).
- Teed, Z. & Deng, J. (2021). Tangent space backpropagation for 3d transformation groups. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 10338–10347.
- Torabi, F. (2021). Imitation learning from observation. PhD thesis, University of Texas at Austin. PhD Thesis.
- Truong, C., Oudre, L., & Vayatis, N. (2020). Selective review of offline change point detection methods. *Signal Processing*, 167, Article 107299.
- Tremblay, J., To, T., Sundaralingam, B., Xiang, Y., Fox, D. & Birchfield, S. (2018). Deep object pose estimation for semantic robotic grasping of household objects. arXiv preprint [arXiv:1809.10790](https://arxiv.org/abs/1809.10790).
- Tyree, S., Tremblay, J., To, T., Cheng, J., Mosier, T., Smith, J. & Birchfield, S. (2022). 6-dof pose estimation of household objects for robotic manipulation: An accessible dataset and benchmark. arXiv preprint [arXiv:2203.05701](https://arxiv.org/abs/2203.05701).
- Torabi, F., Warnell, G. & Stone, P. (2018). Behavioral cloning from observation. In: *Proceedings of the 27th International Joint Conference on Artificial Intelligence*, pp. 4950–4957.
- Torabi, F., Warnell, G. & Stone, P. (2019). Generative adversarial imitation from observation. In: *Imitation, Intent, and Interaction (I3) Workshop at ICML 2019*.
- Torabi, F., Warnell, G. & Stone, P. (2019). Imitation learning from video by leveraging proprioception. In: *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, pp. 3585–3591.
- Vecerik, M., Doersch, C., Yang, Y., Davchev, T., Aytaç, Y., Zhou, G., Hadsell, R., Agapito, L. & Scholz, J. (2023). Robotap: Tracking arbitrary points for few-shot visual imitation. arXiv preprint [arXiv:2308.15975](https://arxiv.org/abs/2308.15975).
- Valassakis, E., Papagiannis, G., Di Palo, N. & Johns, E. (2022). Demonstrate once, imitate immediately (dome): Learning visual servoing for one-shot imitation learning. In: *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 8614–8621. IEEE.
- Wang, D., Devin, C., Cai, Q.-Z., Yu, F. & Darrell, T. (2019). Deep object-centric policies for autonomous driving. In: *2019 International Conference on Robotics and Automation (ICRA)*, pp. 8853–8859. IEEE.
- Wang, C., Fan, L., Sun, J., Zhang, R., Fei-Fei, L., Xu, D., Zhu, Y. & Anandkumar, A. (2023). Mimicplay: Long-horizon imitation learning by watching human play. arXiv preprint [arXiv:2302.12422](https://arxiv.org/abs/2302.12422).
- Wen, B., Lian, W., Bekris, K., & Schaal, S. (2022). You only demonstrate once: Category-level manipulation from single visual demonstration. arXiv preprint [arXiv:2201.12716](https://arxiv.org/abs/2201.12716).
- Wen, C., Lin, X., So, J., Chen, K., Dou, Q., Gao, Y. & Abbeel, P. (2023). Any-point trajectory modeling for policy learning. arXiv preprint [arXiv:2401.00025](https://arxiv.org/abs/2401.00025).
- Wen, B., Yang, W., Kautz, J. & Birchfield, S. (2024). Foundationpose: Unified 6d pose estimation and tracking of novel objects. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 17868–17879.
- Xu, J., Cheng, W., Gao, Y., Wang, X., Gao, S. & Shan, Y. (2024). Instantmesh: Efficient 3d mesh generation from a single image with sparse-view large reconstruction models. arXiv preprint [arXiv:2404.07191](https://arxiv.org/abs/2404.07191).
- Xiong, H., Li, Q., Chen, Y.-C., Bharadhwaj, H., Sinha, S. & Garg, A. (2021). Learning by watching: Physical imitation of manipulation skills from human videos. In: *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 7827–7834. IEEE.
- Xu, M., Xu, Z., Chi, C., Veloso, M. & Song, S. (2023). Xskill: Cross embodiment skill discovery. In: *Conference on Robot Learning*, pp. 3536–3555. PMLR.
- Zhu, Y., Joshi, A., Stone, P. & Zhu, Y. (2022). Viola: Imitation learning for vision-based manipulation with object proposal priors. arXiv preprint [arXiv:2210.11339](https://arxiv.org/abs/2210.11339).
- Zhu, Y., Jiang, Z., Stone, P. & Zhu, Y. (2023). Learning generalizable manipulation policies with object-centric 3d representations. In: *7th Annual Conference on Robot Learning*.
- Zhang, R., Lee, S., Hwang, M., Hiranaka, A., Wang, C., Ai, W., Tan, J.J.R., Gupta, S., Hao, Y., Levine, G., et al. (2023). Noir: Neural signal operated intelligent robots for everyday activities. arXiv preprint [arXiv:2311.01454](https://arxiv.org/abs/2311.01454).
- Zhou, Q.-Y., Park, J. & Koltun, V. (2018). Open3d: A modern library for 3d data processing. arXiv preprint [arXiv:1801.09847](https://arxiv.org/abs/1801.09847).
- Zhu, Y., Tremblay, J., Birchfield, S. & Zhu, Y. (2021). Hierarchical planning for long-horizon manipulation with geometric and symbolic scene graphs. In: *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 6541–6548. IEEE.
- Zhu, Y., Wong, J., Mandlekar, A., Martín-Martín, R., Joshi, A., Nasiriany, S., Zhu, Y. & Lin, K. (2020). robosuite: A modular simulation framework and benchmark for robot learning. In: arXiv Preprint [arXiv:2009.12293](https://arxiv.org/abs/2009.12293).

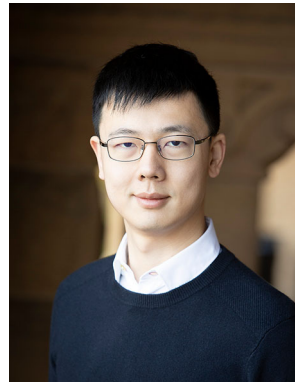
Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Yifeng Zhu received his Ph.D. in Computer Science at the University of Texas at Austin. He has also spent time at Carnegie Mellon University as a visiting student.



Arisrei Lim received a BS in Computer Science and a BS in Mathematics at the University of Texas at Austin. He is currently a research engineer at Reve, working on controllable visual generation.



Yuke Zhu Associate Professor in the Department of Computer Science at the University of Texas at Austin and the director of the Robot Perception and Learning (RPL) Lab. He is also a Director and Distinguished Research Scientist at NVIDIA Research, where he co-leads the Generalist Embodied Agent Research (GEAR) group.



Peter Stone holds the Truchard Foundation Chair in Computer Science at the University of Texas at Austin. He is Chair of the Computer Science Department, as well as Founding Director of Texas Robotics. In 2013 he was awarded the University of Texas System Regents' Outstanding Teaching Award and in 2014 he was inducted into the UT Austin Academy of Distinguished Teachers, earning him the title of University Distinguished Teaching Professor. Professor Stone's

research interests in Artificial Intelligence include machine learning (especially reinforcement learning), multiagent systems, and robotics. Professor Stone received his Ph.D in Computer Science in 1998 from Carnegie Mellon University. From 1999 to 2002 he was a Senior Technical Staff Member in the Artificial Intelligence Principles Research Department at AT&T Labs - Research. He is an Alfred P. Sloan Research Fellow, Guggenheim Fellow, AAAI Fellow, IEEE Fellow, AAAS Fellow, ACM Fellow, Fulbright Scholar, and 2004 ONR Young Investigator. In 2007 he received the prestigious IJCAI Computers and Thought Award, given biannually to the top AI researcher under the age of 35, in 2016 he was awarded the ACM/SIGAI Autonomous Agents Research Award, and in 2024 he was awarded the ACM/AAAI Allen Newell Award. Professor Stone co-founded Cogitai, Inc., a startup company focused on continual learning, in 2015, and currently serves as Chief Scientist of Sony AI.